

USO INTERNO · CONFIDENCIAL

Sistema de Gerenciamento de Estoque Interno

Documentação Técnica Completa — Arquitetura, Banco de Dados, Segurança, Design de Interface, Termos de Uso e Política de Dados

Versão do documento: 2.1 (com tutorial de desenvolvimento passo a passo)

Data: Agosto de 2026

Plataforma: Aplicação Desktop — Electron + Node.js + MySQL

Autor: Equipe de Engenharia de Software / TI

Classificação: Documento Interno — Não deve ser compartilhado fora da empresa

⚠ Este documento contém informações técnicas e operacionais sensíveis. Distribuição restrita aos colaboradores autorizados.

Sumário

1. Introdução e Visão Geral	4
2. Objetivos, Escopo e Público-Alvo	5
3. Termos de Uso e Termo de Responsabilidade	6
4. Dados Utilizados e Política de Privacidade Interna (LGPD)	8
5. Stack Tecnológica Detalhada	10
6. Arquitetura de Software	12
7. Sistema de Design (UI/UX) e Identidade Visual	15
8. Modelo de Dados (MySQL) — DDL Completo e ER	18
9. Segurança da Informação	23
10. Perfis de Usuário, RBAC e Catálogo de Permissões	26
11. Módulos do Sistema (detalhamento funcional completo)	29
12. Fluxos de Processo	38
13. Integração com Scanner de Código de Barras/QR	41
14. Geração de Etiquetas	43
15. Exportação de Relatórios	45
16. Estratégia de Testes	47
17. Requisitos Não-Funcionais	49
18. Auditoria, Logs e Compliance	51
19. Backup e Recuperação de Desastres	53
20. Estrutura de Pastas e Padrões de Código	54
21. CI/CD e Empacotamento	56
22. Plano de Implementação (Roadmap)	58
23. Treinamento e Onboarding de Usuários	59
24. Glossário de Termos	61
25. Anexos	62
26. Guia de Desenvolvimento Passo a Passo (Tutorial Completo)	66
27. Considerações Finais	86

* Numeração de página aproximada — gerada automaticamente na exportação final.

1. Introdução e Visão Geral

Este documento é a especificação técnica de referência do **Sistema de Gerenciamento de Estoque Interno**, doravante referido também como "**GWM ASTHA**" (nome de trabalho adotado para fins de identidade visual do produto, conforme Seção 7). Trata-se de uma aplicação **desktop**, construída em **Electron**, para uso exclusivamente interno em um computador dedicado, fisicamente instalado na sala de estoque/almojarifado da empresa, sem exposição à internet pública e sem previsão de acesso remoto.

Esta é a **revisão 2.1** do documento. Em relação à versão 1.0, a revisão 2.0 aprofundou cada módulo técnico com contratos de API completos (canais IPC, payloads, respostas, códigos de erro), adicionou o Sistema de Design da interface (Seção 7), formalizou os **Termos de Uso** e a **Política de Dados Utilizados** (Seções 3 e 4) — exigidos por se tratar de um sistema que trata dados pessoais de colaboradores —, detalhou a estratégia de testes, o modelo de ameaças de segurança (STRIDE), o plano de treinamento de usuários e incluiu um glossário e anexos com exemplos reais de payloads JSON e entradas de log de auditoria. A revisão 2.1, atual, adiciona o **Guia de Desenvolvimento Passo a Passo** (Seção 26) — um tutorial operacional completo com 20 passos sequenciais, cada um com Definição de Pronto, testes padrão de indústria e testes específicos de segurança e dados.

O sistema resolve um problema extremamente comum em ambientes industriais e operacionais: o controle de estoque feito em planilhas soltas, sem rastreabilidade de quem retirou o quê, sem trilha de auditoria, sem alerta de estoque mínimo e sem identificação física padronizada dos itens. O GWM ASTHA substitui esse modelo por um banco de dados relacional (MySQL) com regras de negócio centralizadas, controle de acesso por cargo (RBAC), geração automática de etiquetas com QR Code para leitura por scanner, e dashboards gerenciais com dados financeiros e operacionais em tempo real.

Por que isso importa na prática: cada retirada de item sem rastreio é um ponto cego de custo. Multiplicado por centenas de itens e dezenas de colaboradores ao longo de um ano, a ausência de controle formal de estoque representa uma das causas mais comuns e mais evitáveis de perda patrimonial em empresas de manufatura e manutenção.

1.1 Natureza do Sistema

Atributo	Definição
Tipo de aplicação	Desktop, monousuário por sessão, multiusuário por instalação (múltiplos colaboradores usam o mesmo terminal com login individual)
Ambiente de execução	Computador local dedicado, localizado na sala de estoque, sem acesso à internet
Banco de dados	MySQL 8.0+, local ou em servidor de rede interna da empresa
Modelo de licenciamento	Software proprietário de uso interno — não é comercializado nem distribuído externamente
Usuários-alvo	Operadores, Técnicos, Analistas, Group Leaders (GL) e Administradores do almoxarifado
Criticidade dos dados	Alta — envolve valores financeiros de estoque, dados pessoais de colaboradores e trilha de auditoria patrimonial

2. Objetivos, Escopo e Público-Alvo

2.1 Objetivos Primários

- Rastreabilidade total:** toda entrada e saída de item deve estar associada a um usuário identificado, data/hora, motivo e (quando aplicável) documento fiscal de origem;
- Controle financeiro do estoque:** visão consolidada do valor total imobilizado em estoque, por categoria, por localização e por período;
- Padronização física:** todo item cadastrado recebe uma etiqueta única com QR Code, permitindo identificação e retirada rápida via leitor de código de barras;
- Governança de acesso:** controle de permissões por cargo, com aprovação obrigatória em múltiplos níveis para ações críticas (remoção de item, reset de senha);
- Auditoria imutável:** toda ação relevante do sistema gera um registro de log que não pode ser alterado ou apagado por nenhum perfil de usuário, incluindo ADM;
- Redução de dependência de planilhas:** eliminar o uso de arquivos Excel soltos como fonte de verdade do estoque.

2.2 Escopo Incluído

- Cadastro e gestão de usuários, cargos, departamentos e hierarquia de liderança;
- Cadastro completo de itens de estoque (consumo e ativo fixo), máquinas/equipamentos e prateleiras;
- Registro de entradas (compra, devolução, ajuste de inventário) e saídas (retirada para manutenção, uso/consumo, empréstimo);
- Fluxo de aprovação para remoção/baixa de itens;
- Geração automática de etiquetas com QR Code/código de barras;
- Visualização de arquivos 3D de itens (.obj, .stl, .3mf) e gestão de manuais técnicos (PDF ou link);
- Dashboard gerencial com KPIs e gráficos;
- Exportação de relatórios em CSV e XLSX;
- Aba "Sobre" com documentação de uso e ficha técnica de construção do software;
- Trilha de auditoria completa de todas as ações relevantes.

2.3 Fora de Escopo (Nesta Versão)

Item fora de escopo	Justificativa
Acesso via web ou aplicativo mobile remoto	É um sistema desktop local por definição de projeto; acesso remoto introduziria superfície de ataque incompatível com o propósito de uso interno isolado
Integração nativa com ERP (SAP, TOTVS, Protheus)	Pode ser avaliada em fase 2, via exportação/importação de arquivos estruturados (CSV/XLSX) ou API dedicada futura
Emissão fiscal de notas fiscais	O sistema registra dados de NF para fins de controle interno, mas não substitui o sistema fiscal homologado da empresa
Multiempresa / múltiplas filiais na mesma instalação	Assume-se uma única unidade de estoque por instância do sistema; expansão multi-filial exigiria redesenho do modelo de dados (adição de tenant_id)
Aplicativo mobile nativo para os operadores	A leitura via scanner USB já cobre a necessidade operacional imediata; app mobile pode ser avaliado como evolução futura

2.4 Público-Alvo deste Documento

Este documento destina-se a: (a) desenvolvedores responsáveis pela construção e manutenção do sistema; (b) times de TI/infraestrutura responsáveis pela instalação, backup e suporte; (c) gestores (GL/ADM) que precisam entender as regras de negócio e permissões; (d) auditoria interna e compliance, para avaliação de conformidade com LGPD e políticas internas de segurança da informação.

3. Termos de Uso e Termo de Responsabilidade

Por se tratar de um sistema corporativo interno que processa dados pessoais de colaboradores e informações patrimoniais da empresa, é obrigatório que todo usuário, antes do primeiro acesso, tome ciência e aceite formalmente os termos abaixo. Recomenda-se implementar esse aceite como uma tela obrigatória de "aceite de termos" no primeiro login, com registro do aceite (usuário, data/hora, versão do termo) na tabela `aceites_termos` (ver Seção 8.2 para o DDL correspondente).

3.1 Termo de Uso Aceitável — Texto de Referência

TERMO DE USO E RESPONSABILIDADE — SISTEMA DE GERENCIAMENTO DE ESTOQUE INTERNO

1. Objeto. O presente termo regula o uso do sistema de gerenciamento de estoque interno ("Sistema"), disponibilizado exclusivamente para uso profissional pelos colaboradores autorizados da empresa, no exercício de suas funções relacionadas ao controle de almoxarifado, estoque e ativos.

2. Natureza do acesso. O acesso ao Sistema é individual, intransferível e vinculado ao crachá funcional e às credenciais (usuário e senha) do colaborador. É expressamente proibido o compartilhamento de senha entre colaboradores, ainda que entre membros da mesma equipe ou hierarquia.

3. Uso aceitável. O colaborador compromete-se a utilizar o Sistema exclusivamente para as finalidades relacionadas às suas atribuições funcionais, incluindo: consulta de estoque, registro de entradas e saídas de itens, cadastro de itens (quando aplicável ao seu cargo), geração de relatórios e demais funcionalidades disponibilizadas conforme seu nível de permissão.

4. Vedações. É expressamente vedado: (a) inserir dados falsos ou incorretos no Sistema, incluindo quantidades, valores, motivos de retirada ou identificação de responsáveis; (b) tentar acessar funcionalidades ou dados para os quais o colaborador não possui permissão, incluindo tentativas de burlar o controle de acesso (RBAC); (c) utilizar credenciais de outro colaborador, inclusive mediante autorização informal deste; (d) extrair, copiar ou compartilhar dados do Sistema (relatórios, dados de estoque, dados de colaboradores) para finalidades alheias ao interesse da empresa ou para terceiros não autorizados; (e) tentar modificar, corromper ou apagar registros do log de auditoria.

5. Monitoramento e auditoria. O colaborador declara ciência de que todas as ações realizadas no Sistema — incluindo login, cadastro, edição, exclusão, movimentação de itens e exportação de relatórios — são registradas em log de auditoria permanente, associado à sua identificação individual, podendo ser utilizadas para fins de apuração de responsabilidade em caso de irregularidade, extravio de patrimônio ou descumprimento das políticas internas da empresa.

6. Responsabilidade por movimentações. O colaborador é responsável pela veracidade e precisão das informações registradas em seu nome, incluindo motivo de retirada, quantidade e destino do item. Movimentações registradas incorretamente devem ser comunicadas imediatamente ao GL ou Analista responsável para correção formal (nunca por alteração direta e não documentada).

7. Segurança de credenciais. O colaborador compromete-se a manter sua senha em sigilo, não anotá-la em locais visíveis, e comunicar imediatamente ao GL/ADM em caso de suspeita de comprometimento de sua conta. A senha poderá ser redefinida exclusivamente por perfis GL ou ADM, mediante solicitação formal do colaborador ou de seu líder direto.

8. Propriedade dos dados. Todos os dados inseridos no Sistema, incluindo cadastros, relatórios e histórico de movimentações, são de propriedade exclusiva da empresa, não cabendo ao colaborador qualquer direito de retenção, cópia ou uso posterior desses dados fora do vínculo empregatício.

9. Sanções. O descumprimento deste termo poderá acarretar, conforme a gravidade apurada em processo interno: advertência formal, suspensão de acesso ao Sistema, e demais medidas disciplinares previstas nas políticas internas de Recursos Humanos e no regulamento interno da empresa, sem prejuízo de eventuais medidas cíveis ou criminais cabíveis em caso de dano patrimonial comprovado.

10. Vigência e aceite. Este termo é apresentado obrigatoriamente no primeiro acesso de cada colaborador e sempre que houver revisão material de seu conteúdo. O aceite é condição obrigatória para uso do Sistema e fica registrado com data, hora e versão do termo aceite.

3.2 Implementação Técnica do Aceite de Termos

Recomenda-se armazenar o texto do termo versionado (para permitir histórico de alterações) e registrar o aceite de forma auditável:

```
CREATE TABLE termos_uso (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  versao      VARCHAR(10) NOT NULL,
  conteudo    TEXT NOT NULL,
  vigente_desde DATETIME NOT NULL,
  criado_em   DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE aceites_termos (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  usuario_id  INT NOT NULL,
  termo_id    INT NOT NULL,
  aceite_em   DATETIME DEFAULT CURRENT_TIMESTAMP,
  ip_maquina  VARCHAR(45) NULL,
  CONSTRAINT fk_aceite_usuario FOREIGN KEY (usuario_id) REFERENCES usuarios(id),
  CONSTRAINT fk_aceite_termo FOREIGN KEY (termo_id) REFERENCES termos_uso(id),
  UNIQUE KEY uk_usuario_termo (usuario_id, termo_id)
) ENGINE=InnoDB;
```

Regra de negócio: no login, o Main Process verifica se existe um registro em `aceites_termos` para o usuário referente à versão vigente de `termos_uso`. Caso não exista, a tela de termos é exibida de forma bloqueante antes de liberar qualquer outra tela do sistema.

4. Dados Utilizados e Política de Privacidade Interna (LGPD)

O Sistema trata dados pessoais de colaboradores (identificação, cargo, histórico de ações) e, portanto, está sujeito à **Lei Geral de Proteção de Dados (Lei nº 13.709/2018 — LGPD)** no que diz respeito ao tratamento de dados de funcionários no contexto da execução do contrato de trabalho e do legítimo interesse da empresa em proteger seu patrimônio.

4.1 Inventário de Dados Pessoais Tratados

Categoria de dado	Campos	Finalidade	Base legal (LGPD)
Identificação funcional	Nome completo, crachá, nome de usuário, departamento, cargo, foto de perfil (opcional)	Identificação individual para controle de acesso e autoria de ações	Execução do contrato de trabalho (art. 7º, V)
Credenciais de acesso	Senha (armazenada apenas como hash Argon2id, nunca em texto plano)	Autenticação no sistema	Execução do contrato de trabalho / legítimo interesse em segurança
Dados de contato	E-mail corporativo (opcional)	Comunicação de notificações do sistema	Execução do contrato de trabalho
Histórico de atividade	Login/logout, IP da máquina (sempre a mesma máquina local), ações realizadas (CRUD, aprovações, exportações)	Auditoria, apuração de responsabilidade, segurança	Legítimo interesse (art. 7º, IX) e cumprimento de obrigação legal/regulatória de controle patrimonial
Dados de movimentação	Itens retirados/inseridos, quantidade, motivo, observações associadas ao usuário	Controle de estoque e rastreabilidade patrimonial	Legítimo interesse / execução do contrato de trabalho

Importante: o Sistema não coleta dados sensíveis na acepção da LGPD (art. 5º, II) — não há campos de saúde, biometria (além de eventual leitura de crachá, que não é dado biométrico), origem racial, convicção religiosa, opinião política, filiação sindical ou orientação sexual. Caso alguma integração futura venha a envolver esses dados, uma nova avaliação de impacto (RIPD) deve ser conduzida antes da implementação.

4.2 Finalidades do Tratamento

- Controle de acesso individualizado ao Sistema, com autenticação e autorização por cargo;
- Rastreabilidade de ações para fins de auditoria patrimonial e apuração de responsabilidade em caso de extravio, dano ou uso indevido de itens de estoque;
- Geração de relatórios gerenciais agregados (que podem, secundariamente, expor volume de atividade por colaborador — ex: "ranking de retiradas por usuário" no Dashboard, usado para identificar padrões operacionais, não para fins de vigilância individual arbitrária);
- Comunicação operacional (notificações de aprovação pendente, reset de senha).

4.3 Compartilhamento de Dados

Os dados tratados pelo Sistema **não são compartilhados com terceiros externos à empresa** em nenhuma hipótese, dado que o Sistema opera de forma isolada, sem conexão à internet e sem integrações externas nesta versão. O acesso interno aos dados segue estritamente a matriz de permissões da Seção 10.

4.4 Retenção e Descarte

Tipo de dado	Prazo de retenção sugerido	Observação
Cadastro de usuário ativo	Durante todo o vínculo empregatício	Desativação (soft-delete) no desligamento, nunca exclusão física, para preservar histórico de auditoria
Cadastro de usuário desligado	Mínimo de 5 anos após o desligamento	Alinhado a boas práticas contábeis/trabalhistas e à necessidade de rastreabilidade patrimonial de longo prazo
Logs de auditoria	Mínimo de 5 anos	Base para eventual apuração de responsabilidade patrimonial ou disciplinar
Movimentações de estoque	Indeterminado (dado histórico/contábil de patrimônio)	Não há "descarte" — dado histórico de patrimônio deve ser preservado integralmente

4.5 Direitos do Titular (Colaborador)

Todo colaborador, na condição de titular de dados pessoais tratados pelo Sistema, tem direito a: (a) confirmar a existência de tratamento de seus dados; (b) acessar os dados tratados a seu respeito, incluindo relatório de todas as ações registradas em seu nome; (c) solicitar correção de dados cadastrais incorretos ou desatualizados (nome, e-mail, departamento); (d) ser informado sobre com quem seus dados são compartilhados (neste caso, apenas internamente, conforme matriz de permissões). Solicitações devem ser direcionadas ao GL/ADM responsável ou ao encarregado de dados (DPO) da empresa, conforme política corporativa de privacidade.

Nota de engenharia: a funcionalidade "Meus Dados" (dentro da aba Perfil, Seção 11.1) deve permitir que o próprio colaborador visualize um resumo de todas as ações registradas em seu nome, operacionalizando o direito de acesso sem depender de solicitação manual ao TI para cada consulta.

4.6 Medidas de Segurança Aplicadas aos Dados Pessoais

- Senhas nunca armazenadas em texto plano — apenas hash Argon2id (Seção 9.1);
- Controle de acesso por cargo (RBAC), restringindo quem visualiza dados de outros colaboradores (Seção 10);
- Log de auditoria com permissão de banco de dados restrita a `INSERT / SELECT`, impedindo alteração retroativa mesmo por erro operacional ou má-fé;
- Backup criptografado em repouso quando armazenado em mídia externa/NAS (Seção 19);
- Isolamento de rede — o computador de estoque não deve ter acesso irrestrito à internet, reduzindo superfície de exfiltração de dados.

5. Stack Tecnológica Detalhada

A seleção de tecnologias segue três critérios: (1) maturidade e adoção de mercado comprovada, evitando dependências experimentais em um sistema que lida com patrimônio da empresa; (2) produtividade de desenvolvimento, priorizando TypeScript e ferramentas com tipagem forte; (3) custo zero de licenciamento, já que se trata de um sistema interno sem orçamento de licenças de terceiros.

Camada	Tecnologia	Versão mínima recomendada	Justificativa
Runtime desktop	Electron	31.x	Padrão de mercado para apps desktop híbridos (VSCode, Slack, Figma Desktop)
Linguagem	TypeScript	5.4+	Tipagem estática reduz bugs em regras de negócio financeiras/patrimoniais
UI Framework	React	18.x	Ecossistema maduro, componentização, grande disponibilidade de mão de obra
Bundler	Vite	5.x	Build e HMR extremamente rápidos comparado a Webpack
Estilo	Tailwind CSS + shadcn/ui	3.x	Consistência visual rápida sem reinventar componentes de UI
Estado global	Zustand	4.x	API simples, menos boilerplate que Redux para o porte do projeto
ORM	Prisma	5.x	Migrations versionadas, type-safety ponta a ponta com o MySQL
Banco de dados	MySQL	8.0+	InnoDB, transações ACID, ecossistema de administração maduro
Hash de senha	argon2 (npm)	0.31+	Argon2id — recomendação atual da OWASP
Validação de schema	Zod	3.x	Validação de payloads IPC em runtime, com inferência de tipos TS
QR Code	qrcode (npm)	1.5+	Biblioteca padrão de fato em Node.js
Código de barras	bwip-js	4.x	Suporte amplo a simbologias (Code128, EAN13, QR)
Visualização 3D	three.js	r165+	Biblioteca 3D web mais madura e documentada do mercado
Exportação Excel	exceljs	4.x	Preserva formatação/estilo — superior ao xlsx puro para relatórios apresentáveis
Exportação CSV	@fast-csv/format	5.x	Leve, rápido, testado em produção por milhares de projetos
Geração de PDF	pdfkit / puppeteer	—	pdfkit para geração programática leve; puppeteer para relatórios ricos em HTML/CSS
Empacotamento	electron-builder	24.x	Gera instalador Windows (NSIS), assinatura de código, updater opcional
Testes unitários	Vitest	1.x	Compatível com Vite, rápido, API similar ao Jest

Testes E2E	Playwright	1.4x+	Suporte oficial a testes de aplicações Electron
Logs	winston + electron-log	—	Rastreabilidade de erros em produção sem servidor central de logs
Cofre de credenciais local	keytar	7.x	Usa Credential Manager do Windows / Keychain do macOS para armazenar chave de sessão
Controle de versão	Git + GitHub (privado) ou GitLab self-hosted	—	Versionamento e histórico de mudanças, mesmo em projeto interno
CI/CD	GitHub Actions / GitLab CI	—	Build automatizado do instalador a cada release

5.1 Dependências e Gestão de Licenças de Terceiros

Todas as bibliotecas listadas utilizam licenças permissivas (MIT, ISC, Apache 2.0), compatíveis com uso interno corporativo sem obrigação de disponibilização de código-fonte a terceiros. Recomenda-se manter um arquivo `THIRD-PARTY-LICENSES.md` gerado automaticamente via `license-checker` (npm) a cada build, e exibi-lo na aba "Sobre" do sistema (Seção 11.2), como boa prática de compliance de propriedade intelectual.

6. Arquitetura de Software

O Electron organiza a aplicação em dois processos isolados por design: o **Main Process** (Node.js, com acesso total ao sistema operacional) e o **Renderer Process** (a interface, executando em um Chromium com sandbox). Toda comunicação entre eles ocorre exclusivamente via **IPC (Inter-Process Communication)**, mediada por um **Preload Script** que expõe apenas uma API controlada via `contextBridge`.

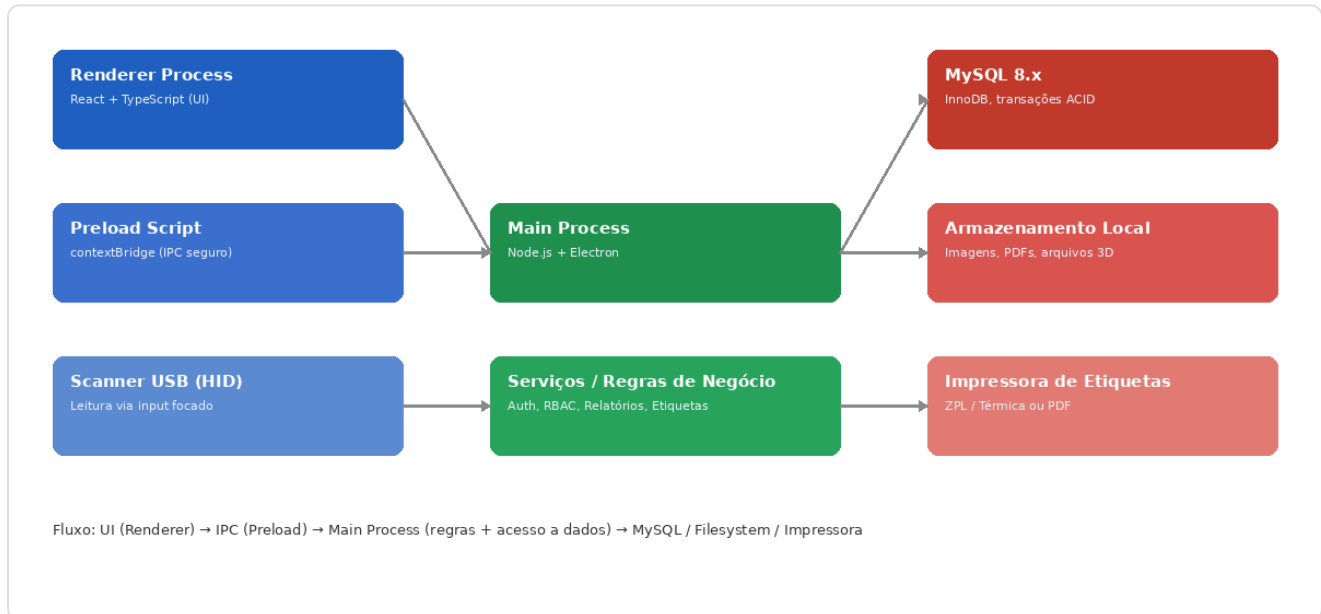


Figura 1 — Fluxo de comunicação entre Renderer, Preload, Main Process, MySQL, armazenamento local e impressora de etiquetas.

6.1 Por que essa separação é inegociável

Se o Renderer tivesse acesso direto ao driver MySQL, qualquer vulnerabilidade de XSS na interface — mesmo vinda de uma dependência de terceiros comprometida — teria caminho direto até o banco de dados, incluindo os hashes de senha de todos os usuários. Com a separação Main/Renderer, a superfície de ataque fica restrita a um conjunto finito e explicitamente whitelisted de canais IPC, cada um validando o payload recebido (via Zod) e a permissão do usuário da sessão ativa antes de tocar em qualquer dado.

6.2 Camadas do Main Process

```
main/
├─ database/      → Prisma Client, connection pool, migrations versionadas
├─ services/      → regras de negócio (auth, estoque, relatórios, etiquetas, dashboard)
├─ ipc/          → handlers IPC (um arquivo por domínio funcional)
├─ security/      → hashing, RBAC middleware, geração/validação de sessão (JWT interno)
├─ printing/      → geração de etiquetas (QR/ZPL) e comunicação com impressora
├─ files/         → gestão de imagens, PDFs de manuais, arquivos 3D no filesystem local
└─ logs/         → configuração winston + audit logger
```

6.3 Catálogo Completo de Canais IPC

Abaixo, o contrato completo de comunicação entre Renderer e Main Process. Cada canal é nomeado no padrão `domínio:acao`, possui um nível mínimo de permissão (conforme Seção 10) e um payload/resposta tipados via Zod/TypeScript.

Canal IPC	Nível mínimo	Payload (entrada)	Resposta
auth:login	Público (pré-sessão)	{ cracha, senha }	{ token, usuario, deveTrocarSenha }
auth:logout	Sessão válida	{ }	{ ok: true }
auth:trocar-senha	Sessão válida	{ senhaAtual, novaSenha }	{ ok: true }
usuarios:listar	GL (4)	{ filtros? }	Usuario[]
usuarios:criar	GL (4)	{ cracha, nome, usuario, departamentold, cargold, liderId? }	Usuario
usuarios:resetar-senha	GL (4)	{ usuariold }	{ senhaTemporaria }
usuarios:desativar	GL (4)	{ usuariold }	{ ok: true }
itens:listar	Operador (1)	{ busca?, tipo?, prateleirald?, page, pageSize }	{ itens: Item[], total }
itens:detalhar	Operador (1)	{ itemId }	ItemDetalhado (com manuais, etiqueta, histórico recente)
itens:criar	Técnico (2)	ItemCreateDTO (ver Seção 11.5.1)	Item + Etiqueta gerada
itens:editar	Técnico (2)	{ itemId, ...camposEditaveis }	Item
itens:solicitar-remocao	Técnico (2)	{ itemId, motivo }	SolicitacaoRemocao
itens:aprovar-remocao	Analista (3)	{ solicitacaold, aprovado, justificativa? }	{ ok: true }
movimentacoes:registrar-saida	Operador (1)	SaidaDTO (ver Seção 11.3.1)	Movimentacao
movimentacoes:registrar-entrada	Técnico (2)*	EntradaDTO	Movimentacao
movimentacoes:listar	Operador (1)*	Filtros (período, item, usuário, tipo)	Movimentacao[]
relatorios:exportar-csv	Analista (3)*	{ tipoRelatorio, filtros }	Buffer / caminho do arquivo salvo
relatorios:exportar-xlsx	Analista (3)*	{ tipoRelatorio, filtros }	Buffer / caminho do arquivo salvo
dashboard:kpis	Analista (3)*	{ periodo? }	{ valorTotal, itensCadastrados, abaixoMinimo, retiradasMes }
etiquetas:reimprimir	Técnico (2)	{ itemId }	{ ok: true }
maquinas:criar	Técnico (2)	MaquinaCreatedTO	Maquina
sistema:configuracoes	ADM (5)	{ ...configs }	{ ok: true }
auditoria:listar	Analista (3) leitura / GL (4) completo	Filtros	LogAuditoria[]

* Operadores e Técnicos possuem acesso parcial/condicional a estes canais conforme detalhado na matriz de permissões da Seção 10 (ex: Operador só lista/exporta suas próprias movimentações).

6.4 Exemplo de Handler IPC com Validação e RBAC

```
// ipc/itens.ipc.ts
import { z } from 'zod';
import { verificarPermissao } from '../security/rbac';
import { criarItem } from '../services/itens.service';

const ItemCreateSchema = z.object({
  nome: z.string().min(2).max(150),
  modelo: z.string().max(100).optional(),
  marca: z.string().max(100).optional(),
  precoUnitario: z.number().nonnegative(),
  tipo: z.enum(['CONSUMO', 'ATIVO_FIXO']),
  prateleiraId: z.number().int().positive(),
  maquinaEquipamentoId: z.number().int().positive().optional(),
  quantidadeInicial: z.number().int().nonnegative(),
  quantidadeMinima: z.number().int().nonnegative(),
  descricao: z.string().max(2000).optional(),
});

ipcMain.handle('itens:criar', async (event, payload, sessao) => {
  verificarPermissao('itens:criar', sessao); // 1. autorização
  const dados = ItemCreateSchema.parse(payload); // 2. validação de schema
  const item = await criarItem(dados, sessao.usuarioId); // 3. regra de negócio
  return item;
});
```

Regra de ouro: nenhum handler IPC deve confiar em flags de permissão vindas do Renderer (ex: `payload.podeEditar`). A verificação de `verificarPermissao()` é sempre a primeira linha do handler, usando exclusivamente os dados da sessão validada no Main Process.

6.5 Modelo de Ameaças (STRIDE) — Resumo

Categoria (STRIDE)	Ameaça considerada	Mitigação aplicada
Spoofing (Falsificação de identidade)	Usuário tentar logar como outro colaborador	Hash Argon2id, bloqueio após 5 tentativas, sessão JWT assinada
Tampering (Adulteração de dados)	Alterar diretamente registros de estoque ou logs via acesso indevido ao banco	Usuário de aplicação com privilégios mínimos no MySQL; logs com permissão apenas INSERT/SELECT
Repudiation (Repúdio de ações)	Colaborador negar ter feito uma retirada	Log de auditoria imutável associado a usuário autenticado, com data/hora
Information Disclosure (Vazamento de dados)	Exposição de hash de senha ou dados de outros usuários	Isolamento Main/Renderer, contextIsolation, RBAC em toda consulta sensível
Denial of Service	Consulta pesada travando a aplicação (ex: exportação de relatório enorme)	Paginação obrigatória, processamento de exportação em processo separado/worker
Elevation of Privilege (Elevação de privilégio)	Operador tentando executar ação de Analista/GL via manipulação do Renderer	RBAC revalidado no Main Process para cada canal IPC, independente do que a UI exhibe

7. Sistema de Design (UI/UX) e Identidade Visual

Esta seção define os **tokens de design**, padrões visuais e diretrizes de interface do GWM ASTHA, baseados na referência visual fornecida pelo time de produto: um painel com fundo em gradiente suave (azul → lavanda → pêssego), cartões translúcidos em estilo "glassmorphism", barra lateral escura de navegação e cartões de KPI com indicadores de tendência. A proposta abaixo adapta essa linguagem visual moderna ao domínio de estoque/almoxxarifado, mantendo legibilidade e contraste adequados para uso prolongado em ambiente industrial.

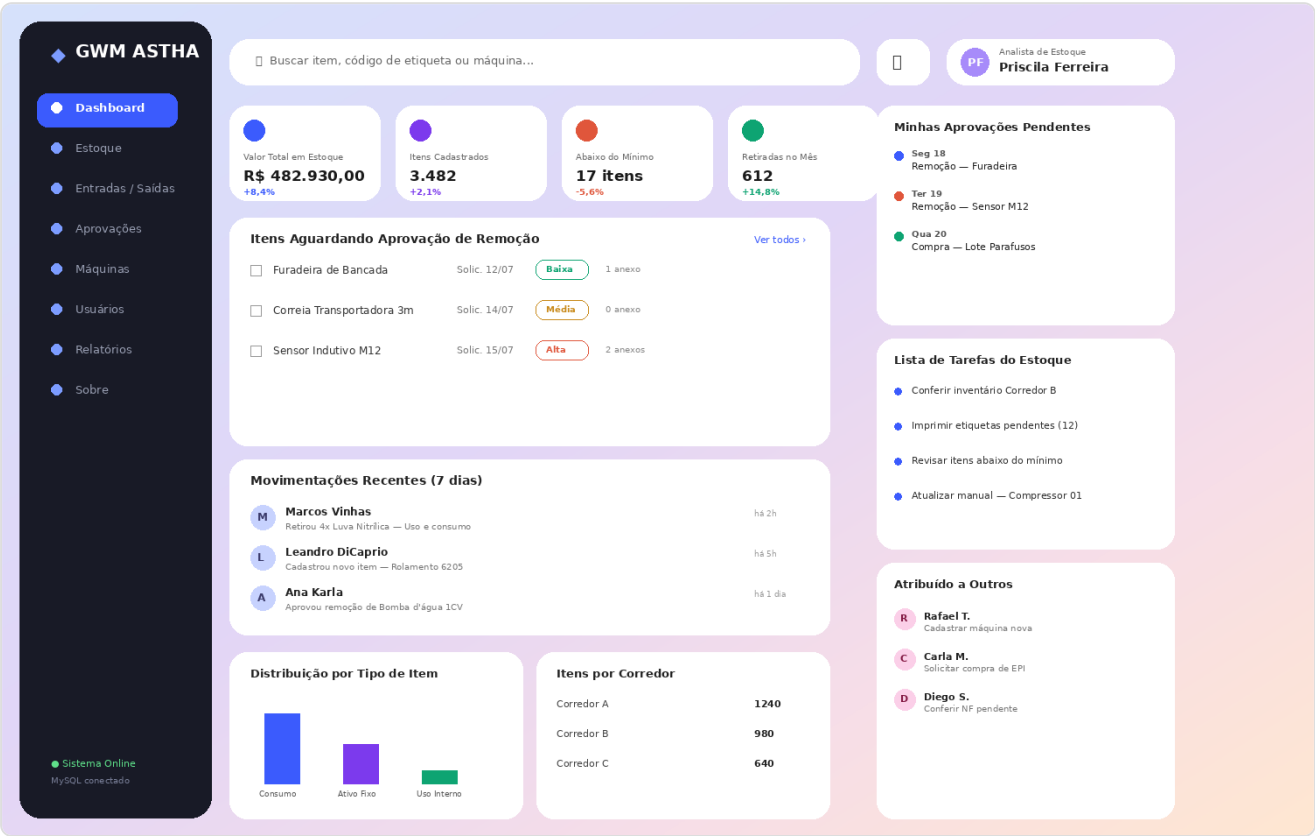


Figura 2 — Aplicação do sistema de design ao módulo de Dashboard (mockup ilustrativo; dados fictícios).

7.1 Paleta de Cores (Design Tokens)

Token	Valor (HEX)	Uso
--bg-gradient-start	#D6E2FC	Início do gradiente de fundo (canto superior esquerdo)
--bg-gradient-mid	#E9D6F7	Meio do gradiente
--bg-gradient-end	#FFE7D1	Fim do gradiente (canto inferior direito)
--sidebar-bg	#151A23	Fundo da barra lateral de navegação
--primary	#3B5BFD	Ações primárias, item de menu ativo, links
--secondary	#7C3AED	Destaques secundários (ativo fixo, badges)
--success	#0EA472	Confirmações, estoque OK, aprovações
--danger	#E0563B	Alertas, estoque abaixo do mínimo, rejeições
--warning	#C98A1A	Pendências, prioridade média
--glass-fill	rgba(255,255,255,0.82)	Preenchimento dos cartões translúcidos

--text-primary	#1A1A1A	Texto principal sobre cartões claros
--text-muted	#6B7280	Texto secundário/legendas

7.2 Tipografia

Uso	Fonte	Peso	Tamanho base
Títulos de seção	Inter / DejaVu Sans	Bold (700)	20-24px
Subtítulos / labels de cartão	Inter	SemiBold (600)	13-14px
Corpo de texto	Inter	Regular (400)	12-13px
Valores de KPI (destaque numérico)	Inter	Bold (700)	22-26px
Texto em tabelas/listas	Inter	Regular (400)	11-12px

7.3 Componentes Visuais Padrão

- **Cartão "glass":** fundo branco translúcido (82% de opacidade), cantos arredondados de 16-20px, borda de 1px branca semi-transparente, leve sombra difusa;
- **Barra lateral (sidebar):** fundo escuro (#151A23), item ativo com pílula azul de destaque (#3B5BFD), ícones simples e consistentes (biblioteca `lucide-react`);
- **Cartão de KPI:** ícone circular colorido à esquerda, label em cinza (text-muted), valor em destaque (bold, 22-26px), indicador de tendência percentual abaixo, colorido conforme positivo/negativo;
- **Badges de prioridade/status:** pílula com borda colorida e texto na mesma cor (baixa=verde, média=laranja, alta=vermelho), seguindo o mesmo padrão usado em "prioridade" de solicitações de remoção;
- **Listas com avatar:** avatar circular com iniciais coloridas para identificação rápida de usuário em listas de atividade recente;
- **Gráficos:** paleta de cores consistente com os tokens acima (azul para consumo, roxo para ativo fixo, verde para indicadores positivos).

7.4 Diretrizes de Acessibilidade Visual

Critério	Diretriz aplicada
Contraste de texto	Mínimo AA (WCAG 2.1) — texto escuro (#1A1A1A) sobre cartões claros garante contraste $\geq 4.5:1$
Tamanho mínimo de fonte	11px para texto de apoio, nunca abaixo disso, considerando uso em monitores de chão de fábrica
Indicação de estado além de cor	Alertas de estoque mínimo usam cor + ícone (⚠) + texto, nunca cor isoladamente
Navegação por teclado	Toda ação crítica (registrar saída, aprovar remoção) deve ser executável via Tab + Enter, sem depender exclusivamente do mouse
Modo alto-contraste opcional	Tema alternativo com fundo sólido (sem gradiente) e cartões com borda mais forte, para ambientes com iluminação difícil ou usuários com baixa visão

7.5 Aplicação da Identidade a Outras Telas

O mesmo padrão visual (fundo em gradiente, sidebar escura, cartões glass) deve ser replicado nas telas de **Estoque** (grade de itens com cartões de produto), **Entradas/Saídas** (formulário com cartão central destacado), **Usuários** (tabela dentro de cartão glass) e **Sobre** (conteúdo textual dentro de um único cartão largo). A tela de **Login** deve usar o mesmo gradiente de fundo, com um único cartão glass centralizado contendo o formulário de autenticação e o logotipo da empresa.

8. Modelo de Dados (MySQL) — DDL Completo e Diagrama ER

O banco de dados segue o padrão de normalização até a 3ª Forma Normal (3FN) nas tabelas transacionais, com desnormalização controlada apenas em campos de auditoria (JSON de `dados_anteriores` / `dados_novos`), onde a flexibilidade de esquema é mais valiosa que a normalização estrita. Charset padrão `utf8mb4` em todas as tabelas, garantindo suporte completo a acentuação e caracteres especiais em observações e nomes.

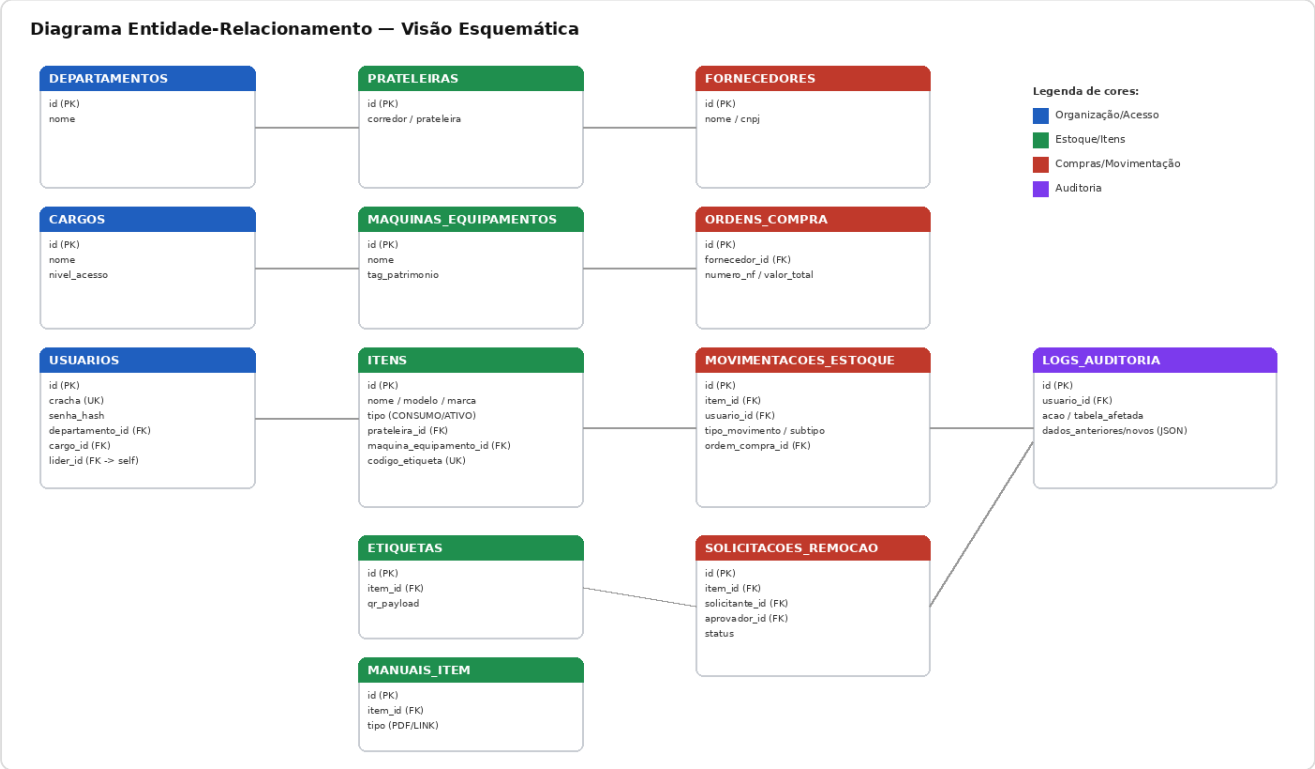


Figura 3 — Diagrama Entidade-Relacionamento esquemático, agrupado por domínio funcional.

8.1 Convenções de Nomenclatura

Elemento	Convenção	Exemplo
Tabelas	snake_case, plural, em português	<code>movimentacoes_estoque</code>
Colunas	snake_case, em português	<code>quantidade_atual</code>
Chaves primárias	sempre <code>id</code> , INT ou BIGINT AUTO_INCREMENT	<code>id</code>
Chaves estrangeiras	<code>{tabela_singular}_id</code>	<code>item_id</code> , <code>usuario_id</code>
Constraints	<code>fk_{tabela}_{referencia}</code>	<code>fk_item_prateleira</code>
Índices	<code>idx_{tabela}_{coluna}</code>	<code>idx_mov_data</code>

8.2 Script DDL Completo

O script abaixo consolida todas as tabelas do sistema, incluindo as tabelas de termos de uso apresentadas na Seção 3.2. Este é o script de referência para a primeira migration (`0001_init.sql` via Prisma Migrate).

```

-- =====
-- Sistema de Gerenciamento de Estoque Interno – GWM ASTHA
-- Script de criação de banco de dados – MySQL 8.0+
-- =====

CREATE DATABASE IF NOT EXISTS estoque_interno
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE estoque_interno;

-- Departamentos
CREATE TABLE departamentos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(100) NOT NULL UNIQUE,
  criado_em DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- Cargos (GL, Analista, Tecnico, Operador, ADM)
CREATE TABLE cargos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(50) NOT NULL UNIQUE,
  nivel_acesso TINYINT NOT NULL COMMENT '1=Operador,2=Tecnico,3=Analista,4=GL,5=ADM',
  descricao VARCHAR(255)
) ENGINE=InnoDB;

INSERT INTO cargos (nome, nivel_acesso, descricao) VALUES
('Operador', 1, 'Retira itens e consulta estoque'),
('Tecnico', 2, 'Cadastra itens e solicita remocoes'),
('Analista', 3, 'Aprova remocoes e acessa relatorios completos'),
('GL', 4, 'Gerencia usuarios e reseta senhas'),
('ADM', 5, 'Acesso total ao sistema');

-- Usuarios
CREATE TABLE usuarios (
  id INT AUTO_INCREMENT PRIMARY KEY,
  cracha VARCHAR(30) NOT NULL UNIQUE,
  nome_completo VARCHAR(150) NOT NULL,
  nome_usuario VARCHAR(50) NOT NULL UNIQUE,
  senha_hash VARCHAR(255) NOT NULL COMMENT 'Argon2id encoded hash',
  departamento_id INT NOT NULL,
  cargo_id INT NOT NULL,
  lider_id INT NULL,
  email VARCHAR(150) NULL,
  foto_perfil VARCHAR(255) NULL,
  ativo BOOLEAN NOT NULL DEFAULT TRUE,
  deve_trocar_senha BOOLEAN NOT NULL DEFAULT TRUE,
  ultimo_login DATETIME NULL,
  tentativas_login TINYINT NOT NULL DEFAULT 0,
  bloqueado_ate DATETIME NULL,
  criado_em DATETIME DEFAULT CURRENT_TIMESTAMP,
  atualizado_em DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_usuario_departamento FOREIGN KEY (departamento_id) REFERENCES departamentos(id),
  CONSTRAINT fk_usuario_cargo FOREIGN KEY (cargo_id) REFERENCES cargos(id),
  CONSTRAINT fk_usuario_lider FOREIGN KEY (lider_id) REFERENCES usuarios(id)
) ENGINE=InnoDB;

CREATE INDEX idx_usuarios_cargo ON usuarios(cargo_id);
CREATE INDEX idx_usuarios_departamento ON usuarios(departamento_id);

-- Fornecedores
CREATE TABLE fornecedores (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(150) NOT NULL,
  cnpj VARCHAR(18) NULL,
  contato_nome VARCHAR(100) NULL,
  telefone VARCHAR(20) NULL,
  email VARCHAR(150) NULL,
  ativo BOOLEAN DEFAULT TRUE

```

```

) ENGINE=InnoDB;

-- Ordens de compra
CREATE TABLE ordens_compra (
    id INT AUTO_INCREMENT PRIMARY KEY,
    numero_pedido VARCHAR(50) NOT NULL,
    fornecedor_id INT NOT NULL,
    numero_nf VARCHAR(50) NULL,
    data_compra DATE NOT NULL,
    valor_total DECIMAL(12,2) NOT NULL DEFAULT 0,
    arquivo_nf VARCHAR(255) NULL,
    criado_em DATETIME DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_ordem_fornecedor FOREIGN KEY (fornecedor_id) REFERENCES fornecedores(id)
) ENGINE=InnoDB;

-- Maquinas e equipamentos
CREATE TABLE maquinas_equipamentos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(150) NOT NULL,
    tag_patrimonio VARCHAR(50) NULL,
    localizacao VARCHAR(150) NULL,
    fabricante VARCHAR(100) NULL,
    modelo VARCHAR(100) NULL,
    ativo BOOLEAN DEFAULT TRUE,
    criado_em DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- Prateleiras
CREATE TABLE prateleiras (
    id INT AUTO_INCREMENT PRIMARY KEY,
    corredor VARCHAR(20) NOT NULL,
    prateleira VARCHAR(20) NOT NULL,
    nivel VARCHAR(20) NULL,
    descricao VARCHAR(150) NULL,
    UNIQUE KEY uk_localizacao (corredor, prateleira, nivel)
) ENGINE=InnoDB;

-- Itens
CREATE TABLE itens (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(150) NOT NULL,
    modelo VARCHAR(100) NULL,
    marca VARCHAR(100) NULL,
    preco_unitario DECIMAL(12,2) NOT NULL DEFAULT 0,
    imagem VARCHAR(255) NULL,
    tipo ENUM('CONSUMO','ATIVO_FIXO') NOT NULL,
    maquina_equipamento_id INT NULL,
    prateleira_id INT NOT NULL,
    descricao TEXT NULL,
    arquivo_3d VARCHAR(255) NULL,
    quantidade_atual INT NOT NULL DEFAULT 0,
    quantidade_minima INT NOT NULL DEFAULT 0,
    codigo_etiqueta VARCHAR(30) NOT NULL UNIQUE,
    ativo BOOLEAN NOT NULL DEFAULT TRUE,
    criado_por INT NOT NULL,
    criado_em DATETIME DEFAULT CURRENT_TIMESTAMP,
    atualizado_em DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_item_maquina FOREIGN KEY (maquina_equipamento_id) REFERENCES maquinas_equipamentos(id),
    CONSTRAINT fk_item_prateleira FOREIGN KEY (prateleira_id) REFERENCES prateleiras(id),
    CONSTRAINT fk_item_criador FOREIGN KEY (criado_por) REFERENCES usuarios(id)
) ENGINE=InnoDB;

CREATE INDEX idx_itens_tipo ON itens(tipo);
CREATE INDEX idx_itens_prateleira ON itens(prateleira_id);
CREATE INDEX idx_itens_codigo ON itens(codigo_etiqueta);
CREATE FULLTEXT INDEX ftx_itens_busca ON itens(nome, modelo, marca, descricao);

-- Manuais do item

```

```

CREATE TABLE manuais_item (
    id INT AUTO_INCREMENT PRIMARY KEY,
    item_id INT NOT NULL,
    titulo VARCHAR(150) NOT NULL,
    tipo ENUM('PDF','LINK') NOT NULL,
    caminho_ou_url VARCHAR(500) NOT NULL,
    criado_em DATETIME DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_manual_item FOREIGN KEY (item_id) REFERENCES itens(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- Etiquetas
CREATE TABLE etiquetas (
    id INT AUTO_INCREMENT PRIMARY KEY,
    item_id INT NOT NULL,
    codigo VARCHAR(30) NOT NULL UNIQUE,
    tipo_etiqueta ENUM('ATIVO_FIXO','CONSUMO') NOT NULL,
    qr_payload VARCHAR(255) NOT NULL,
    impresso BOOLEAN DEFAULT FALSE,
    gerado_por INT NOT NULL,
    gerado_em DATETIME DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_etiqueta_item FOREIGN KEY (item_id) REFERENCES itens(id),
    CONSTRAINT fk_etiqueta_usuario FOREIGN KEY (gerado_por) REFERENCES usuarios(id)
) ENGINE=InnoDB;

-- Movimentacoes de estoque
CREATE TABLE movimentacoes_estoque (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    item_id INT NOT NULL,
    tipo_movimento ENUM('ENTRADA','SAIDA') NOT NULL,
    subtipo ENUM('COMPRA','DEVOLUCAO','RETIRADA_MANUTENCAO','USO_CONSUMO',
        'EMPRESTIMO','AJUSTE_INVENTARIO','BAIXA_REMOCAO') NOT NULL,
    quantidade INT NOT NULL,
    usuario_id INT NOT NULL,
    ordem_compra_id INT NULL,
    observacoes TEXT NULL,
    data_hora DATETIME DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_mov_item FOREIGN KEY (item_id) REFERENCES itens(id),
    CONSTRAINT fk_mov_usuario FOREIGN KEY (usuario_id) REFERENCES usuarios(id),
    CONSTRAINT fk_mov_ordem FOREIGN KEY (ordem_compra_id) REFERENCES ordens_compra(id)
) ENGINE=InnoDB;
CREATE INDEX idx_mov_item ON movimentacoes_estoque(item_id);
CREATE INDEX idx_mov_data ON movimentacoes_estoque(data_hora);
CREATE INDEX idx_mov_usuario ON movimentacoes_estoque(usuario_id);

-- Solicitacoes de remocao
CREATE TABLE solicitacoes_remocao (
    id INT AUTO_INCREMENT PRIMARY KEY,
    item_id INT NOT NULL,
    solicitante_id INT NOT NULL,
    motivo TEXT NOT NULL,
    status ENUM('PENDENTE','APROVADO','REJEITADO') NOT NULL DEFAULT 'PENDENTE',
    aprovador_id INT NULL,
    justificativa_resposta TEXT NULL,
    data_solicitacao DATETIME DEFAULT CURRENT_TIMESTAMP,
    data_resposta DATETIME NULL,
    CONSTRAINT fk_remocao_item FOREIGN KEY (item_id) REFERENCES itens(id),
    CONSTRAINT fk_remocao_solicitante FOREIGN KEY (solicitante_id) REFERENCES usuarios(id),
    CONSTRAINT fk_remocao_aprovador FOREIGN KEY (aprovador_id) REFERENCES usuarios(id)
) ENGINE=InnoDB;

-- Logs de auditoria (somente INSERT/SELECT para o usuario de aplicacao)
CREATE TABLE logs_auditoria (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    usuario_id INT NULL,
    acao VARCHAR(100) NOT NULL,
    tabela_afetada VARCHAR(100) NULL,
    registro_id INT NULL,

```

```

    dados_anteriores JSON NULL,
    dados_novos JSON NULL,
    ip_maquina VARCHAR(45) NULL,
    data_hora DATETIME DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_log_usuario FOREIGN KEY (usuario_id) REFERENCES usuarios(id)
) ENGINE=InnoDB;
CREATE INDEX idx_log_usuario ON logs_auditoria(usuario_id);
CREATE INDEX idx_log_data ON logs_auditoria(data_hora);
CREATE INDEX idx_log_acao ON logs_auditoria(acao);

-- Termos de uso e aceites (Secao 3)
CREATE TABLE termos_uso (
    id INT AUTO_INCREMENT PRIMARY KEY,
    versao VARCHAR(10) NOT NULL,
    conteudo TEXT NOT NULL,
    vigente_desde DATETIME NOT NULL,
    criado_em DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE aceites_termos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    usuario_id INT NOT NULL,
    termo_id INT NOT NULL,
    aceite_em DATETIME DEFAULT CURRENT_TIMESTAMP,
    ip_maquina VARCHAR(45) NULL,
    CONSTRAINT fk_aceite_usuario FOREIGN KEY (usuario_id) REFERENCES usuarios(id),
    CONSTRAINT fk_aceite_termo FOREIGN KEY (termo_id) REFERENCES termos_uso(id),
    UNIQUE KEY uk_usuario_termo (usuario_id, termo_id)
) ENGINE=InnoDB;

```

8.3 Estratégia de Índices

Todos os índices foram definidos considerando os padrões de consulta esperados: busca por código de etiqueta (leitura via scanner, alta frequência), filtro por período em movimentações (relatórios), e busca textual em itens (tela de estoque). O índice `FULLTEXT` em `itens` permite buscas naturais como "parafuso m6 inox" sem depender de `LIKE '%...%'`, que não utiliza índice e degrada em tabelas grandes.

8.4 Estratégia de Migrations

Recomenda-se Prisma Migrate para versionamento de schema. Cada alteração estrutural gera um arquivo de migration numerado e datado, nunca `ALTER TABLE` manual direto em produção. Fluxo recomendado: alteração no `schema.prisma` → `prisma migrate dev` em ambiente local → revisão em Pull Request → `prisma migrate deploy` no ambiente de produção, com backup obrigatório imediatamente antes.

9. Segurança da Informação

9.1 Hashing de Senha — Padrão de Mercado

A recomendação oficial do **OWASP Password Storage Cheat Sheet** é **Argon2id** como primeira escolha — algoritmo vencedor da Password Hashing Competition (2015) — com bcrypt como alternativa aceitável apenas quando Argon2 não está disponível na stack. Em Node.js isso não é uma limitação: usamos o pacote `argon2` (bindings nativos, mantido ativamente).

```
import argon2 from 'argon2';

// Cadastro/reset de senha
const hash = await argon2.hash(senhaPlana, {
  type: argon2.argon2id,
  memoryCost: 19456, // ~19 MB - recomendacao OWASP 2024/2025
  timeCost: 2,
  parallelism: 1,
});

// Validacao de login
const valido = await argon2.verify(usuario.senha_hash, senhaDigitada);
```

Argon2id já embute o salt no próprio hash gerado — não é necessário nem recomendado gerenciar salt manualmente em coluna separada. Nunca usar MD5, SHA1 ou SHA256 puro para senha: são rápidos demais e permitem ataques de força bruta em escala trivial com hardware moderno (GPUs).

9.2 Política de Senha e Bloqueio de Conta

Regra	Valor
Tamanho mínimo	8 caracteres (recomendado 10+)
Complexidade mínima	Combinação de letras e números; validação de força exibida no cadastro
Tentativas antes do bloqueio	5 tentativas incorretas consecutivas
Duração do bloqueio	15 minutos (campo <code>bloqueado_ate</code>)
Senha resetada por GL/ADM	Sempre marca <code>deve_trocar_senha = true</code> , forçando troca no próximo login
Reversibilidade	Nenhuma senha é reversível — nem o ADM "vê" a senha original de outro usuário
Expiração periódica	Opcional — recomenda-se 180 dias em ambientes de alta rotatividade de crachá, configurável em <code>sistema:configuracoes</code>

9.3 Sessão Local

Como o aplicativo é desktop e local, um fluxo OAuth2 completo seria over-engineering. A abordagem adotada:

- Login válido → Main Process gera um **JWT interno**, assinado com chave simétrica gerada na primeira instalação e armazenada via `keytar` (cofre de credenciais nativo do SO — Credential Manager no Windows);
- O JWT carrega `usuario_id`, `cargo`, `nivel_acesso` e expiração curta (8h, equivalente a um turno de trabalho);
- Cada handler IPC sensível valida o JWT ativo antes de executar a ação;

4. Logout limpa o token da sessão em memória no Main Process — nunca fica em `localStorage` do Renderer.

9.4 Configurações de Segurança do Electron

Configuração	Valor	Motivo
<code>contextIsolation</code>	true	Isola o contexto JS da página do contexto do preload
<code>nodeIntegration</code>	false	Renderer nunca acessa Node.js diretamente
<code>sandbox</code>	true	Restringe o processo renderer às permissões mínimas do SO
<code>webSecurity</code>	true	Mantém same-origin policy ativa
CSP (Content-Security-Policy)	Restritiva, sem <code>unsafe-inline</code>	Mitiga XSS mesmo em app local
Atualização de dependências	Dependabot/Renovate configurado no repositório	Evita rodar em produção com biblioteca com CVE conhecida

9.5 Checklist de Segurança (baseado em OWASP ASVS, nível 2 adaptado)

#	Item	Status recomendado
V2.1	Senhas armazenadas apenas como hash (Argon2id)	Obrigatório
V2.2	Política de bloqueio de conta após tentativas falhas	Obrigatório
V3.1	Sessão expira automaticamente após período de inatividade	Obrigatório
V4.1	Controle de acesso revalidado no backend (Main Process) a cada ação	Obrigatório
V5.1	Toda entrada de usuário validada via schema (Zod) antes de uso	Obrigatório
V7.1	Logs de auditoria não podem ser alterados/apagados pela aplicação	Obrigatório
V9.1	Comunicação IPC restrita a canais explicitamente whitelisted	Obrigatório
V14.1	Dependências de terceiros monitoradas quanto a CVEs conhecidas	Recomendado

10. Perfis de Usuário, RBAC e Catálogo de Permissões

O sistema define cinco cargos com níveis de acesso hierárquicos e cumulativos: **Operador (1)**, **Técnico (2)**, **Analista (3)**, **GL — Group Leader (4)** e **ADM (5)**. Conforme especificado no briefing do produto, GL, Analista e ADM possuem as permissões mais amplas; exclusivamente GL e ADM podem resetar senha de terceiros (e a própria).

10.1 Matriz de Permissões por Funcionalidade

Permissão	Operador	Técnico	Analista	GL	ADM
Visualizar estoque	Sim	Sim	Sim	Sim	Sim
Registrar saída de item (retirada)	Sim	Sim	Sim	Sim	Sim
Registrar entrada de item	Parcial (limitado)	Sim	Sim	Sim	Sim
Cadastrar novo item	Não	Sim	Sim	Sim	Sim
Editar item existente	Não	Sim (próprios cadastros)	Sim	Sim	Sim
Solicitar remoção de item	Não	Sim	Sim	Sim	Sim
Aprovar/rejeitar remoção	Não	Não	Sim	Sim	Sim
Ver Dashboard completo	Não (visão básica)	Parcial	Sim	Sim	Sim
Exportar relatórios (CSV/XLSX)	Não	Parcial (próprias movimentações)	Sim	Sim	Sim
Cadastrar usuários	Não	Não	Não	Sim	Sim
Resetar senha de terceiros	Não	Não	Não	Sim	Sim
Configurações do sistema	Não	Não	Não	Parcial	Sim
Gerenciar cargos/ departamentos	Não	Não	Não	Não	Sim
Acesso a logs de auditoria completos	Não	Não	Parcial (leitura)	Sim	Sim

Legenda: **Sim** = acesso total · **Parcial** = acesso parcial/condicional · **Não** = sem acesso

10.2 Implementação Técnica

```
// security/rbac.ts
const PERMISSOES: Record<string, number> = {
  'itens:criar': 2,           // nível mínimo: Técnico
  'itens:aprovar-remocao': 3, // nível mínimo: Analista
  'usuarios:resetar-senha': 4, // nível mínimo: GL
  'sistema:configuracoes': 5, // nível mínimo: ADM
};
```

```
export function verificarPermissao(canal: string, sessao: SessaoUsuario) {
  const nivelNecessario = PERMISSOES[canal];
  if (nivelNecessario && sessao.nivel_acesso < nivelNecessario) {
    throw new ErroPermissaoNegada(`Acao '${canal}' requer nivel ${nivelNecessario}`);
  }
}
```

10.3 Regras Especiais de Negócio do RBAC

- **Operador — Entrada limitada:** pode registrar apenas o subtipo **DEVOLUCAO** (devolução de item que ele mesmo retirou anteriormente); os demais subtipos de entrada exigem nível Técnico ou superior;
- **Técnico — Edição restrita:** só pode editar itens cujo **criado_por** seja ele mesmo, ou itens sem edições prévias por outro cargo superior — evita conflito de "quem pode mexer no cadastro de quem";
- **GL — Reset de senha:** pode resetar senha de qualquer usuário com nível de acesso ≤ 4 (não pode resetar senha de outro ADM, apenas o próprio ADM pode fazer isso por si ou solicitar a outro ADM);
- **ADM — Superusuário:** único perfil com acesso a **sistema:configuracoes** completo, incluindo parâmetros de backup, impressora padrão e política de expiração de senha.

11. Módulos do Sistema — Detalhamento Funcional Completo

11.1 Módulo de Autenticação e Perfil de Usuário

11.1.1 Tela de Login

Campos: crachá ou nome de usuário + senha. Sem opção de recuperação automática por e-mail (o app é local, sem servidor de e-mail) — a tela exibe apenas a orientação "Esqueceu sua senha? Procure seu GL ou o ADM do estoque". Após o aceite dos Termos de Uso (Seção 3) no primeiro acesso, o sistema segue diretamente para a tela principal.

11.1.2 Aba "Meu Perfil"

- Edição de nome, e-mail, foto de perfil (upload de imagem, redimensionada para 256×256px);
- Troca de senha (exige senha atual + nova senha + confirmação, com validação de força);
- Preferências: tema (claro/escuro/alto-contraste), tamanho de fonte, notificação sonora ao concluir retirada via scanner, idioma (PT-BR único nesta versão);
- **"Meus Dados" (LGPD):** resumo de todas as ações registradas em nome do usuário nos últimos 90 dias, operacionalizando o direito de acesso descrito na Seção 4.5.

11.1.3 Aba "Gerenciar Usuários" (GL/ADM)

Listagem com filtro por departamento, cargo e status (ativo/inativo). Formulário de cadastro:

Campo	Tipo	Validação
Crachá	Texto	Obrigatório, único, 3–30 caracteres
Nome completo	Texto	Obrigatório, mín. 3 caracteres
Nome de usuário	Texto	Obrigatório, único, sem espaços
Departamento	Seleção	Obrigatório
Cargo	Seleção	Obrigatório
Líder de grupo	Seleção (usuário existente)	Opcional
E-mail	Texto	Opcional, formato válido se preenchido

Ao salvar, o sistema gera uma senha temporária aleatória (12 caracteres alfanuméricos + símbolo), exibida **uma única vez** em tela para o GL/ADM repassar pessoalmente ao colaborador. O campo `deve_trocar_senha` é automaticamente marcado como verdadeiro.

Resetar senha: o botão "Resetar Senha" (disponível apenas para GL/ADM, conforme matriz da Seção 10) segue o mesmo fluxo — gera senha temporária, exibe uma única vez, força troca no próximo login, e grava log de auditoria `RESET_SENHA` com o ID de quem executou a ação.

Desativação de usuário: sempre soft-delete (`ativo = false`) — nunca exclusão física, pois isso quebraria a integridade referencial do histórico de movimentações e logs de auditoria associados a esse usuário.

11.2 Módulo "Sobre" / Documentação Embutida

Aba estática com: versão do software e changelog; manual de uso resumido (renderizado a partir de Markdown embutido); ficha técnica de construção (stack, versão do Node/Electron, licenças de terceiros conforme Seção 5.1); diagrama de arquitetura (Figura 1); dados de contato do time de TI responsável pela manutenção; e o texto vigente dos Termos de Uso (Seção 3), sempre consultável.

11.3 Módulo de Entradas e Saídas

11.3.1 Registro de Saída

Busca de item por nome, código de etiqueta (compatível com leitura via scanner — ver Seção 13) ou navegação por categoria/prateleira. Payload de exemplo enviado ao canal `movimentacoes:registrar-saida`:

```
{
  "itemId": 1042,
  "quantidade": 2,
  "subtipo": "RETIRADA_MANUTENCAO",
  "responsavelRetirada": "usuario_id_ou_nome_livre",
  "observacoes": "Substituicao de correia da esteira 03 - OS #4521"
}
```

Regras de validação: quantidade deve ser maior que zero e menor ou igual à `quantidade_atual` do item (validado com `SELECT ... FOR UPDATE` — ver Seção 17.1 sobre concorrência); campo `observacoes` obrigatório quando o subtipo é `EMPRESTIMO` (para registrar prazo/condição de devolução esperada).

11.3.2 Registro de Entrada

Tipos disponíveis: **Compra** (vinculada a uma ordem de compra — número de pedido, NF, fornecedor, data, valor), **Devolução** e **Ajuste de Inventário** (usado em conferências físicas periódicas, exige justificativa obrigatória e é sempre registrado com o log de auditoria mais detalhado, por ser a movimentação mais suscetível a divergência).

```
{
  "itemId": 1042,
  "quantidade": 50,
  "subtipo": "COMPRA",
  "ordemCompra": {
    "numeroPedido": "PC-2026-0134",
    "fornecedorId": 8,
    "numeroNf": "NF-55810",
    "dataCompra": "2026-07-28",
    "valorTotal": 1250.00
  },
  "observacoes": "Reposicao trimestral - almoxarifado central"
}
```

11.3.3 Relatórios de Movimentação

Filtros disponíveis: período (data inicial/final), item, usuário responsável, departamento, tipo de movimento (entrada/saída), subtipo. Colunas exibidas: data/hora, item, quantidade, tipo, subtipo, usuário responsável, observações, valor calculado (quantidade x preço unitário vigente no momento da movimentação — **importante**: o valor é congelado no momento do registro, não recalculado retroativamente se o preço do item mudar depois, para preservar a fidelidade histórica do relatório).

11.4 Módulo Dashboard



Figura 4 — Estrutura funcional do Dashboard (wireframe de referência para posicionamento de componentes).

Cartões de KPI: **valor total em estoque** (`SUM(quantidade_atual * preco_unitario)` de itens ativos); **total de itens cadastrados ativos**; **itens abaixo do estoque mínimo** (alerta vermelho clicável, filtra a lista); **total de retiradas no mês corrente**.

Gráficos e relatórios: movimentação de estoque nos últimos 6 meses (entradas vs. saídas); top 10 itens mais retirados no período; giro de estoque por categoria (consumo vs. ativo fixo); ranking de usuários por volume de retiradas (útil para identificar padrões operacionais incomuns); distribuição de valor por prateleira/corredor (apoio ao inventário físico); lista de alertas de estoque mínimo com ação rápida "Gerar pedido de compra".

Otimização recomendada: materializar os agregados mais pesados (ex: valor total) em uma tabela `dashboard_cache`, recalculada via trigger a cada movimentação ou job agendado a cada 5 minutos, evitando recalcular `SUM()` sobre toda a tabela de itens a cada abertura de tela. Torna-se essencial quando o catálogo ultrapassar ~50 mil itens.

11.5 Módulo de Estoque (Itens, Máquinas, Prateleiras)

11.5.1 Cadastro de Item — Especificação Completa de Campos

Campo	Tipo	Obrigatório	Validação/Observação
Nome do item	Texto	Sim	2-150 caracteres
Modelo	Texto	Opcional	até 100 caracteres
Marca (fabricante)	Texto	Opcional	até 100 caracteres
Preço unitário	Decimal	Sim	≥ 0, usado no cálculo do Dashboard
Imagem	Upload (jpg/png/webp)	Opcional	máx. 5MB, redimensionada para thumbnail 512×512

Tipo	Consumo / Ativo Fixo	Sim	Define regras de baixa/depreciação
Máquina/Equipamento vinculado	Seleção (com cadastro rápido inline)	Opcional	Vínculo N:1 com maquinas_equipamentos
Prateleira	Seleção (corredor/prateleira/nível)	Sim	Endereço físico único
Descrição	Texto longo	Opcional	até 2000 caracteres
Arquivo 3D	Upload (.obj/.stl/.3mf)	Opcional	máx. 30MB, visualizador embutido (Seção 11.7)
Manuais	Upload PDF ou Link	Opcional	múltiplos por item (Seção 11.8)
Quantidade inicial	Inteiro	Sim	≥ 0, gera movimentação ENTRADA automática
Quantidade mínima	Inteiro	Sim	≥ 0, base para alertas do Dashboard

Ao salvar, o sistema gera automaticamente: (1) um **codigo_etiqueta** único no formato **EST-XXXXXX**; (2) um registro em **etiquetas** com o QR/barcode correspondente; (3) uma movimentação de **ENTRADA** inicial; (4) um log de auditoria **CRIAR_ITEM** com o payload completo em **dados_novos**.

11.5.2 Cadastro de Máquinas/Equipamentos

Aba própria para pré-cadastrar o parque de máquinas (nome, tag de patrimônio da própria máquina, fabricante, modelo, localização física), permitindo que, no cadastro de item, o Técnico apenas vincule o item à máquina já existente — por exemplo, "Filtro de óleo" vinculado a "Compressor de Ar 01".

11.5.3 Cadastro de Prateleiras

Estrutura corredor/prateleira/nível, gerando um "endereço físico" único por item — essencial para inventário físico e conferência rápida em campo.

11.6 Geração de Etiquetas — Visão Funcional

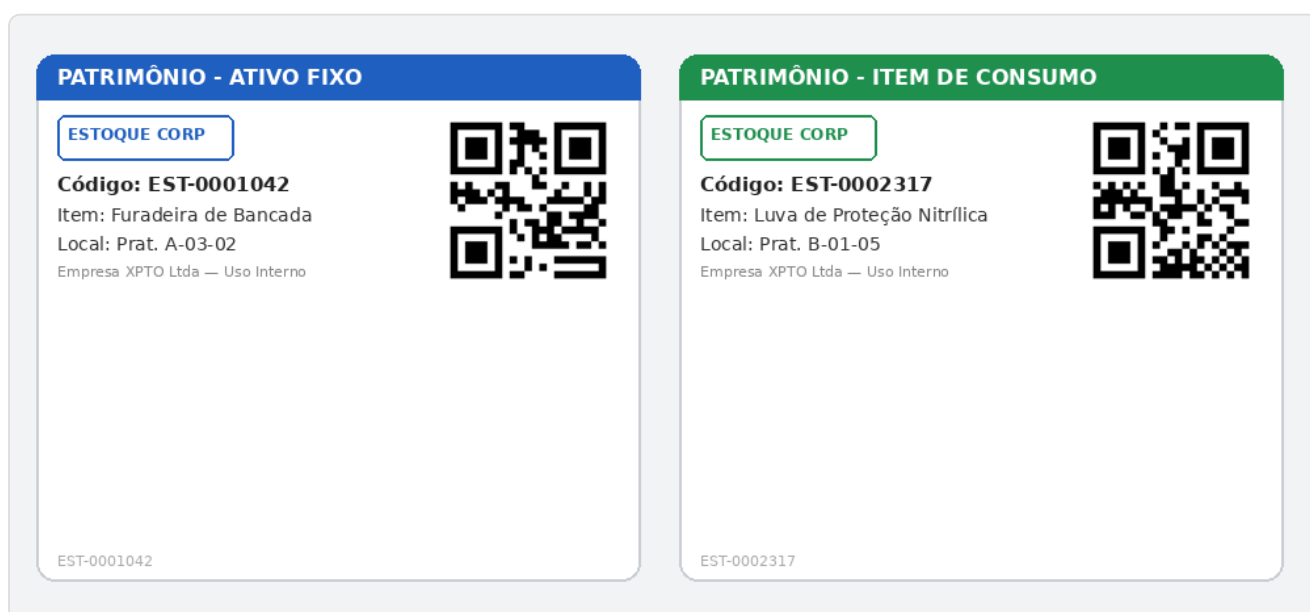


Figura 5 — Mockup ilustrativo de etiqueta de Ativo Fixo (azul) e Item de Consumo (verde), com QR Code contendo o código único do item.

Detalhamento técnico completo (dimensões, protocolo ZPL, impressoras suportadas) na Seção 14.

11.7 Visualizador de Arquivos 3D

Ao clicar no item na tela de estoque, se houver arquivo 3D vinculado, abre-se um modal com visualização interativa (rotação, zoom, pan) usando **three.js**:

```
import { OBJLoader } from 'three/examples/jsm/loaders/OBJLoader.js';
import { STLLoader } from 'three/examples/jsm/loaders/STLLoader.js';
import { ThreeMFLoader } from 'three/examples/jsm/loaders/3MFLoader.js';

function carregarModelo(caminho: string, extensao: string, scene: THREE.Scene) {
  const loaders: Record<string, any> = { obj: OBJLoader, stl: STLLoader, '3mf': ThreeMFLoader };
  const Loader = loaders[extensao];
  new Loader().load(caminho, (objeto) => scene.add(objeto));
}
```

Diferença importante entre formatos: **.stl** é geometria pura (sem cor/material); **.3mf** e **.obj** (com **.mtl** associado) suportam cor e material embutidos, resultando em visualização mais fiel. Arquivos 3D ficam armazenados em **storage/modelos-3d/{item_id}/** no sistema de arquivos local — nunca como BLOB no MySQL, que não é adequado para arquivos binários grandes.

11.8 Gestão de Manuais (PDF / Links)

Cada item pode ter múltiplos manuais associados. **Upload de PDF:** arquivo copiado para **storage/manuais/{item_id}/**, nome sanitizado, visualização embutida via **pdf.js** sem depender de aplicativo externo. **Link externo:** URL para o site do fabricante — preferível quando o manual original é extenso ou protegido por direitos autorais do fabricante, evitando redistribuição de material de terceiros.

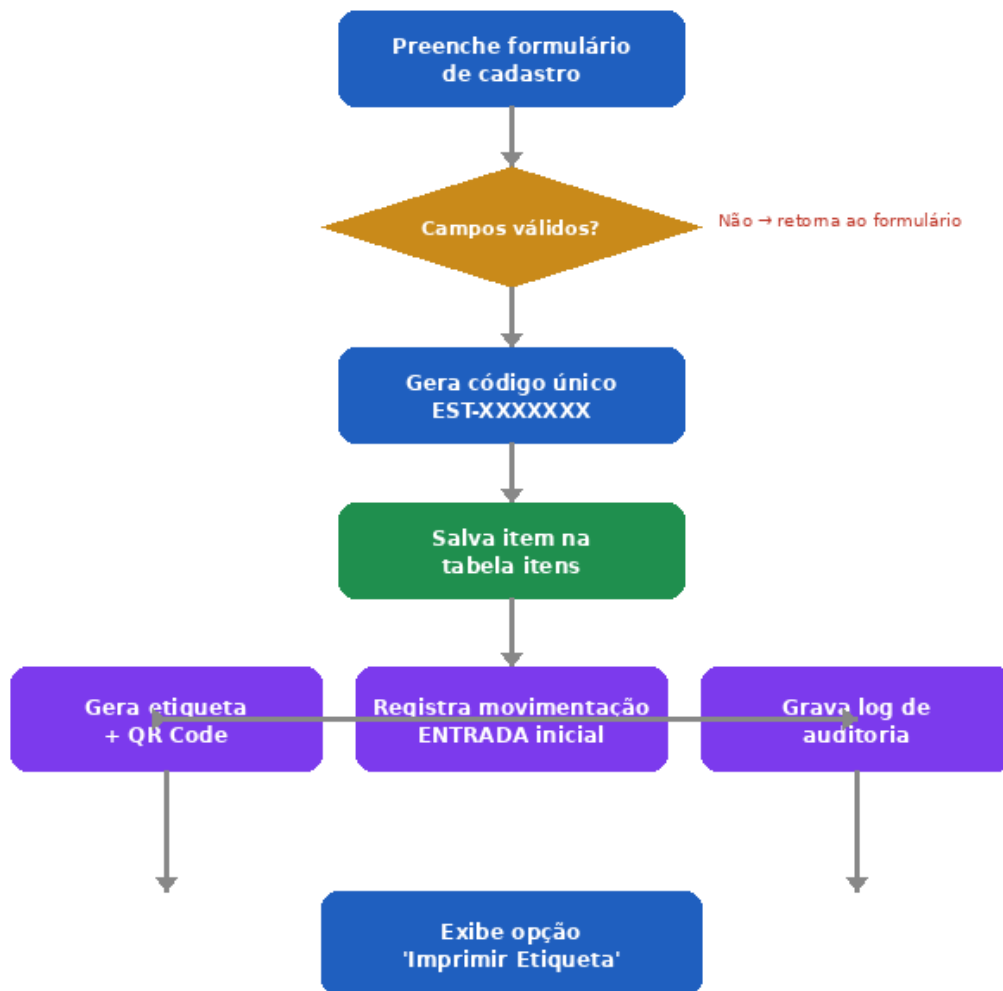
11.9 Aprovação de Remoção de Itens

Fluxo obrigatório para dar baixa em qualquer item, detalhado com fluxograma na Seção 12.3. Resumo funcional: Técnico ou superior solicita remoção com motivo obrigatório → status **PENDENTE** → notificação in-app para Analista/GL/ADM → decisão (aprovação gera **item.ativo = false** + movimentação **BAIXA_REMOCAO**; rejeição exige justificativa) → notificação ao solicitante do resultado.

12. Fluxos de Processo

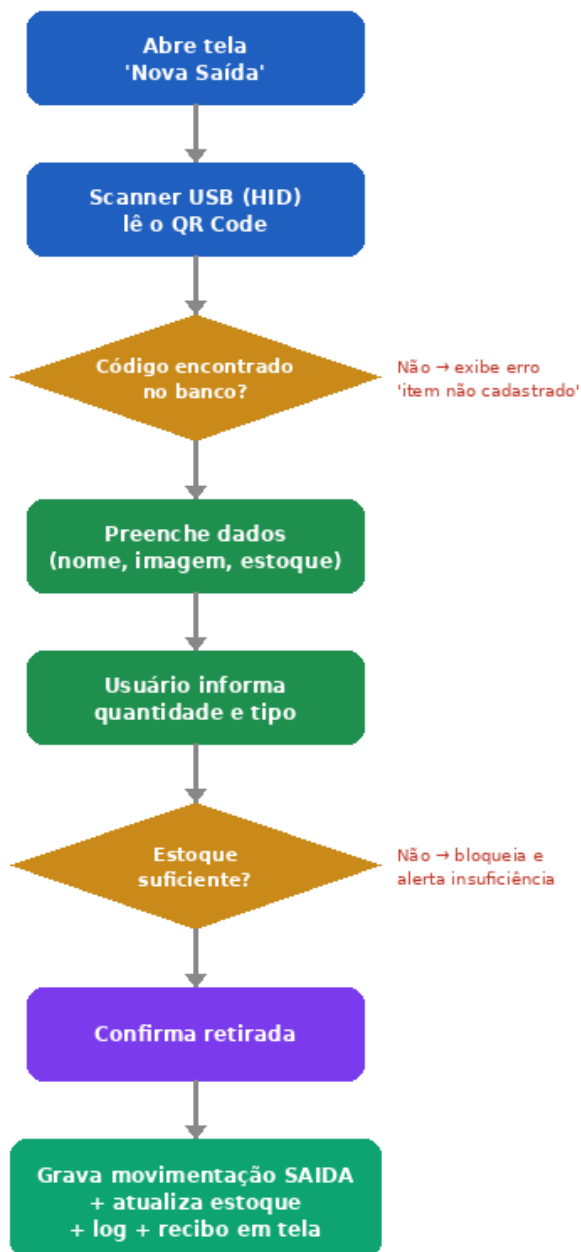
12.1 Cadastro de Item → Geração de Etiqueta

Fluxo — Cadastro de Item e Geração de Etiqueta



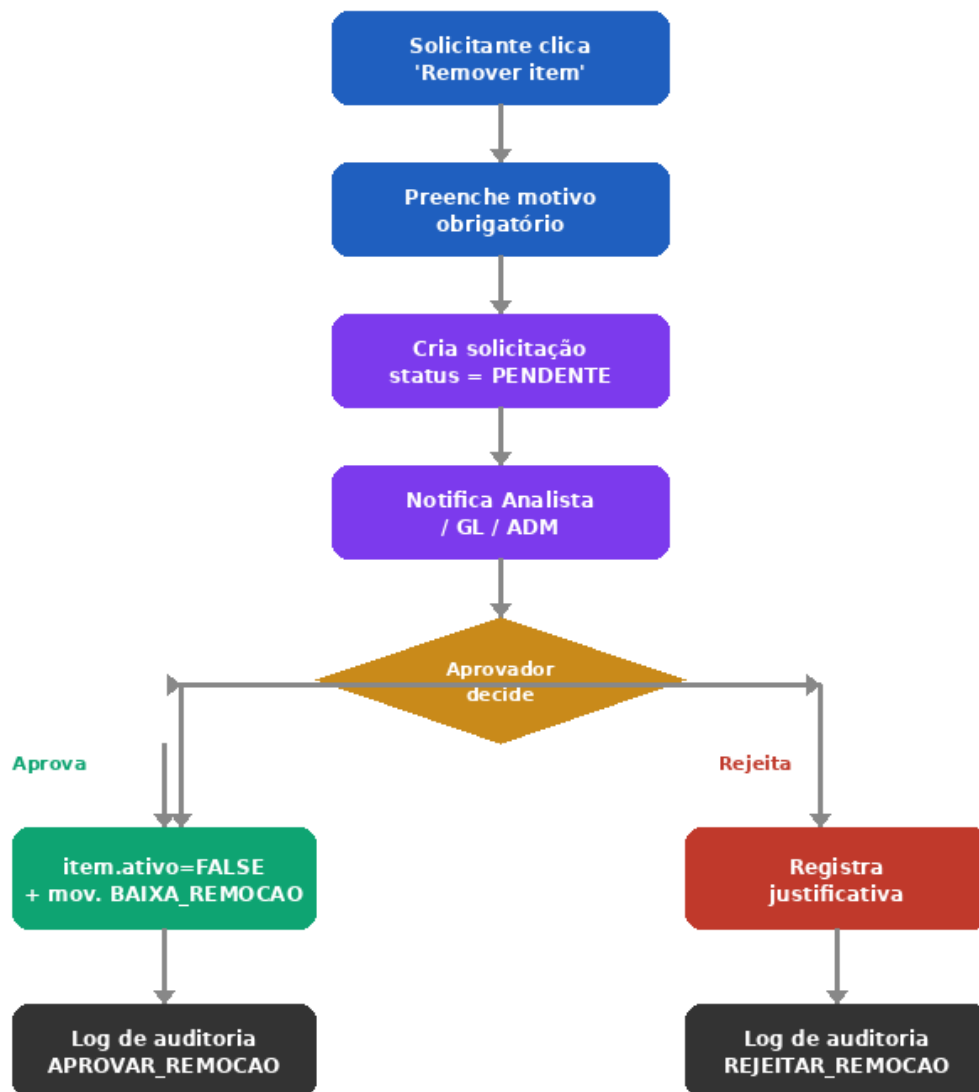
12.2 Retirada de Item via Scanner

Fluxo — Retirada de Item via Scanner



12.3 Solicitação e Aprovação de Remoção

Fluxo — Solicitação e Aprovação de Remoção



12.4 Fluxo Complementar — Entrada por Compra

1. Técnico/Analista/GL/ADM abre "Nova Entrada" → seleciona subtipo "Compra";
2. Sistema pergunta se a ordem de compra já existe (busca por número de pedido) ou se deve ser criada;
3. Se nova: preenche fornecedor, número de pedido, número da NF, data e valor total, com opção de anexar PDF da NF;
4. Seleciona item e quantidade recebida;
5. Sistema grava `movimentacoes_estoque` (ENTRADA/COMPRA), vinculada à `ordens_compra`, atualiza `quantidade_atual` e grava log de auditoria.

12.5 Fluxo Complementar — Reset de Senha

1. Colaborador esquece a senha → procura pessoalmente o GL ou ADM (não há fluxo de autoatendimento por e-mail, por design);

2. GL/ADM acessa "Gerenciar Usuários" → localiza o colaborador → clica em "Resetar Senha";
3. Sistema gera senha temporária aleatória, exibida uma única vez na tela do GL/ADM;
4. GL/ADM repassa a senha pessoalmente ao colaborador (nunca por canal digital não seguro, conforme boa prática interna);
5. No próximo login, o campo `deve_trocar_senha = true` força a troca imediata antes de liberar qualquer outra tela;
6. Log de auditoria `RESET_SENHA` é gravado, associando quem executou o reset e quem foi o beneficiário.

13. Integração com Scanner de Código de Barras/QR

A grande maioria dos leitores USB de código de barras/QR do mercado (Honeywell Voyager 1200g, Zebra DS2208, Motorola/Symbol LS2208) opera em modo **HID Keyboard Wedge**: do ponto de vista do sistema operacional, o scanner é um teclado. Ele "digita" o conteúdo lido seguido de um **Enter**. Isso significa que **não é necessário nenhum driver especial nem biblioteca de baixo nível**.

13.1 Implementação

```
// hooks/useScannerListener.ts
export function useScannerListener(onCodigoLido: (codigo: string) => void) {
  useEffect(() => {
    let buffer = '';
    let ultimoEvento = Date.now();
    const handler = (e: KeyboardEvent) => {
      const agora = Date.now();
      // scanners digitam muito mais rapido que humanos (<30ms entre teclas)
      if (agora - ultimoEvento > 100) buffer = '';
      ultimoEvento = agora;

      if (e.key === 'Enter') {
        if (buffer.startsWith('EST-')) onCodigoLido(buffer);
        buffer = '';
      } else if (e.key.length === 1) {
        buffer += e.key;
      }
    };
    window.addEventListener('keydown', handler);
    return () => window.removeEventListener('keydown', handler);
  }, [onCodigoLido]);
}
```

A checagem de intervalo entre teclas (`agora - ultimoEvento > 100`) é uma proteção adicional para diferenciar leitura automática de scanner (tipicamente <30ms entre caracteres) de digitação manual acidental, reduzindo falsos positivos.

13.2 Modelos de Scanner Recomendados e Testados no Mercado

Modelo	Tipo	Observação
Honeywell Voyager 1200g	USB com fio, HID	Referência de mercado, leitura robusta de QR e barras 1D
Zebra DS2208	USB com fio, HID	Bom custo-benefício, amplamente usado em varejo/almoxarifado
Motorola/Symbol LS2208	USB com fio, HID (apenas 1D)	Não lê QR Code — usar apenas se a etiqueta usar código de barras 1D
Inateck BCST-70	Bluetooth, modo HID	Alternativa sem fio, mesma integração por ser HID

13.3 Cenário Futuro — Scanner Serial/Não-HID

Caso a empresa opte por um leitor serial ou Bluetooth que não emule teclado, a integração exigiria a biblioteca `serialport` (Node.js) rodando no Main Process, com leitura direta da porta e repasse via IPC ao Renderer. Esta é

uma exceção arquitetural — não deve ser implementada preventivamente, seguindo o princípio de engenharia "não construa o que você ainda não precisa".

13.4 Troubleshooting Comum

Sintoma	Causa provável	Solução
Scanner não gera nenhum caractere	Campo de input não está em foco	Garantir foco automático no campo de leitura ao abrir a tela "Nova Saída"
Código lido incompleto	QR Code impresso em baixa resolução/qualidade	Verificar configuração de impressão (Seção 14.4) e distância de leitura
Sistema não reconhece o prefixo EST-	Etiqueta antiga ou de outro sistema	Validar se o item foi cadastrado neste sistema; reimprimir etiqueta se necessário

14. Geração de Etiquetas

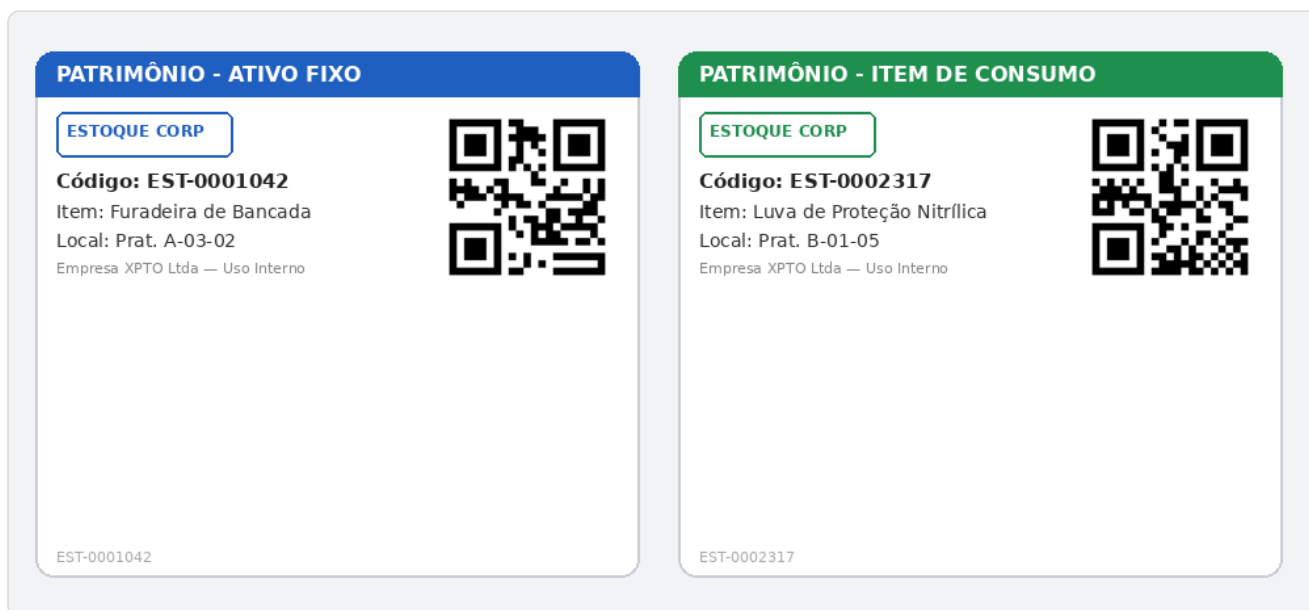


Figura 6 — Etiquetas de Ativo Fixo e Item de Consumo (mockup ilustrativo; paleta final a critério do time de identidade visual da empresa).

14.1 Especificação Física

Parâmetro	Recomendação
Dimensão — itens pequenos	40×20mm
Dimensão — itens médios/grandes	50×30mm
Material	Adesivo poliéster (resistente a óleo/graxa) para ativo fixo; papel couché adesivo para consumo
Correção de erro do QR Code	Nível M (15% de tolerância) — equilíbrio entre densidade e robustez em ambiente industrial

14.2 Conteúdo da Etiqueta

- Cabeçalho colorido por tipo (azul = Ativo Fixo, verde = Item de Consumo — paleta ilustrativa, ajustável à identidade visual da empresa);
- QR Code contendo o payload `EST-XXXXXXX` (idêntico ao `codigo_etiqueta`), lido pelo scanner no momento da retirada;
- Nome do item, código legível por humano, localização (prateleira);
- Rodapé com identificação da empresa / "Uso Interno".

14.3 Geração do QR Code

```
import QRCode from 'qrcode';

export async function gerarPayloadEtiqueta(item: Item) {
  const payload = `EST-${item.codigoEtiqueta}`;
```

```
const qrDataUrl = await QRCode.toDataURL(payload, { errorCorrectionLevel: 'M' });
return { payload, qrDataUrl };
}
```

14.4 Impressão — Impressoras Térmicas (ZPL)

Para impressoras térmicas de etiqueta (Zebra GC420t, Elgin L42 Pro, Argox OS-214), o sistema monta o comando ZPL diretamente — mais confiável que depender de driver de impressão genérico do Windows:

```
^XA
^F030,30^BQN,2,4^FDMA,EST-0001042^FS
^F0220,40^A0N,28,28^FD Furadeira de Bancada^FS
^F0220,80^A0N,20,20^FD Prat. A-03-02^FS
^F0220,110^A0N,18,18^FDEST-0001042^FS
^XZ
```

14.5 Impressão — Impressora Comum (Folha Adesiva)

Alternativa para empresas sem impressora térmica dedicada: geração de PDF com layout de folha adesiva padrão (ex: Pimaco 6180, 10 etiquetas por folha A4), usando `pdfkit` para posicionar múltiplas etiquetas na grade correta da folha.

14.6 Reimpressão

Disponível via canal `etiquetas:reimprimir` (nível mínimo Técnico), útil quando a etiqueta física se deteriora. A reimpressão reutiliza o mesmo `codigo_etiqueta` — nunca gera um código novo para um item já existente, preservando a integridade do histórico de movimentações vinculado àquele código.

15. Exportação de Relatórios (CSV / XLSX)

Toda tela de relatório (Entradas/Saídas, Dashboard, Auditoria) oferece exportação em dois formatos, salvos via diálogo nativo do sistema operacional (`dialog.showSaveDialog` do Electron), sem depender de download via navegador.

15.1 CSV

```
import { writeToString } from '@fast-csv/format';

export async function exportarCSV(dados: MovimentacaoRelatorio[]) {
  const csv = await writeToString(dados, { headers: true });
  return '\uFEFF' + csv; // BOM evita acentuacao quebrada no Excel do Windows
}
```

15.2 XLSX

```
import ExcelJS from 'exceljs';

export async function exportarXLSX(dados: MovimentacaoRelatorio[]) {
  const wb = new ExcelJS.Workbook();
  const ws = wb.addWorksheet('Movimentacoes');
  ws.columns = [
    { header: 'Data/Hora', key: 'dataHora', width: 20 },
    { header: 'Item', key: 'item', width: 30 },
    { header: 'Tipo', key: 'tipo', width: 12 },
    { header: 'Quantidade', key: 'quantidade', width: 12 },
    { header: 'Usuario', key: 'usuario', width: 25 },
    { header: 'Valor Total', key: 'valorTotal', width: 15, style: { numFmt: 'R$ #,##0.00' } },
  ];
  ws.getRow(1).font = { bold: true };
  dados.forEach((d) => ws.addRow(d));
  return wb.xlsx.writeBuffer();
}
```

15.3 Convenção de Nomenclatura de Arquivo

Padrão obrigatório: `relatorio_{tipo}_{AAAAAMDD}_{HHmm}.{ext}`, por exemplo `relatorio_movimentacoes_20260803_1432.xlsx`, garantindo rastreabilidade de quando o relatório foi gerado mesmo após ser compartilhado internamente.

15.4 Tipos de Relatório Disponíveis

Relatório	Perfil mínimo	Conteúdo
Movimentações (entradas/saídas)	Analista (Técnico: apenas próprias)	Detalhamento completo por período/item/usuário
Estoque atual (posição)	Analista	Snapshot de todos os itens ativos, quantidade e valor
Itens abaixo do mínimo	Analista	Lista de reposição sugerida
Auditoria de ações	GL (Analista: leitura)	Log completo de ações por usuário/período

Solicitações de remoção	Analista	Histórico de aprovações/rejeições
-------------------------	----------	-----------------------------------

16. Estratégia de Testes

16.1 Pirâmide de Testes Adotada

Camada	Ferramenta	Cobertura-alvo	Foco
Testes unitários	Vitest	≥ 80% em services/	Regras de negócio isoladas (cálculo de estoque, validações Zod, RBAC)
Testes de integração	Vitest + banco MySQL de teste (Docker)	Principais fluxos de handlers IPC	Interação real com o banco via Prisma
Testes E2E	Playwright (suporte oficial a Electron)	Fluxos críticos de usuário	Login, cadastro de item, retirada via scanner simulado, aprovação de remoção

16.2 Exemplo de Teste Unitário

```
// services/__tests__/movimentacoes.service.test.ts
import { describe, it, expect } from 'vitest';
import { validarQuantidadeSaida } from '../movimentacoes.service';

describe('validarQuantidadeSaida', () => {
  it('deve rejeitar quantidade maior que o estoque disponivel', () => {
    expect(() => validarQuantidadeSaida({ quantidadeAtual: 5, quantidadeSolicitada: 10 }))
      .toThrow('Estoque insuficiente');
  });

  it('deve aceitar quantidade igual ao estoque disponivel', () => {
    expect(() => validarQuantidadeSaida({ quantidadeAtual: 5, quantidadeSolicitada: 5 }))
      .not.toThrow();
  });
});
```

16.3 Exemplo de Teste E2E (Playwright + Electron)

```
// e2e/retirada-item.spec.ts
import { test, expect, _electron as electron } from '@playwright/test';

test('operador consegue registrar retirada de item via codigo', async () => {
  const app = await electron.launch({ args: ['.'] });
  const window = await app.firstWindow();

  await window.fill('#input-cracha', '00123');
  await window.fill('#input-senha', 'senha-teste');
  await window.click('#btn-login');

  await window.click('#nav-nova-saida');
  await window.fill('#input-scanner', 'EST-0001042\n'); // simula leitura do scanner
  await window.fill('#input-quantidade', '1');
  await window.selectOption('#select-subtipo', 'USO_CONSUMO');
  await window.click('#btn-confirmar-saida');

  await expect(window.locator('#msg-sucesso')).toBeVisible();
  await app.close();
});
```

16.4 Testes Obrigatórios Antes de Cada Release

- Fluxo completo de login, incluindo bloqueio após 5 tentativas incorretas;
- Cadastro de item com geração de etiqueta e verificação do QR Code gerado;
- Retirada de item com validação de estoque insuficiente;
- Fluxo completo de solicitação e aprovação/rejeição de remoção;
- Exportação de relatório CSV e XLSX com verificação de integridade do arquivo gerado;
- Verificação de que nenhum canal IPC crítico é acessível sem sessão válida (teste de segurança negativo).

17. Requisitos Não-Funcionais

Categoria	Requisito
Performance	Busca de item por código deve responder em < 200ms mesmo com 100 mil itens cadastrados (garantido pelos índices da Seção 8.3)
Disponibilidade	Aplicação funciona 100% offline, dependendo apenas da disponibilidade local do MySQL
Concorrência	Suporte a múltiplas estações usando o mesmo MySQL em rede interna, com transações InnoDB evitando condição de corrida
Usabilidade	Fluxo de retirada via scanner executável em menos de 10 segundos por item
Portabilidade	Instalador único .exe (NSIS), sem exigir instalação manual de Node/MySQL pelo usuário final
Acessibilidade	Navegação por teclado, contraste WCAG AA nas telas principais (Seção 7.4)
Internacionalização	PT-BR único nesta versão, arquitetura preparada via i18next para expansão futura
Escalabilidade de dados	Suporte projetado para até 200 mil itens e 5 milhões de registros de movimentação sem degradação perceptível
Tempo de inicialização	Aplicação pronta para uso em até 5 segundos após abertura (splash screen com carregamento assíncrono)

17.1 Concorrência — Exemplo Crítico

```
START TRANSACTION;
SELECT quantidade_atual FROM itens WHERE id = ? FOR UPDATE;
-- validacao de quantidade suficiente na aplicacao
UPDATE itens SET quantidade_atual = quantidade_atual - ? WHERE id = ?;
INSERT INTO movimentacoes_estoque (...) VALUES (...);
COMMIT;
```

O bloqueio pessimista (**FOR UPDATE**) evita o clássico cenário de dois operadores retirando o último item simultaneamente e o estoque ficando negativo — um bug simples de descrever, mas caro de corrigir depois que já causou divergência de inventário real.

17.2 Requisitos Mínimos de Hardware

Configuração mínima para execução da estação de estoque em condições aceitáveis, para catálogos de até ~20 mil itens ativos e uso por 1-2 usuários simultâneos na mesma máquina.

Componente	Mínimo	Observação
Processador (CPU)	Intel Core i3 (8ª geração ou superior) / AMD Ryzen 3 3000 series	Electron + Chromium embutido consomem mais CPU no boot que apps nativos leves
Memória RAM	8 GB	4 GB é tecnicamente possível, mas gera lentidão perceptível com o visualizador 3D aberto
Armazenamento	10 GB livres (SSD recomendado)	Cobre aplicação, MySQL local, imagens, manuais PDF e arquivos 3D de até ~2.000 itens
Placa de vídeo (GPU)	Integrada (Intel UHD / AMD Vega) com suporte a WebGL 2.0	Necessário para o visualizador 3D (three.js) funcionar sem travar

Portas USB	2x USB-A (1 para scanner, 1 para impressora, se não for de rede)	USB 2.0 já é suficiente para scanner HID e impressora térmica
Monitor	1366×768	Resolução mínima para o layout do Dashboard não quebrar (recomenda-se 1920×1080)
Rede	Ethernet ou Wi-Fi (apenas se o MySQL estiver em servidor de rede interna)	Dispensável se o MySQL rodar localmente na mesma máquina

17.3 Requisitos Recomendados de Hardware (Produção / Catálogo Grande)

Configuração recomendada para instalações com catálogo acima de 20 mil itens, múltiplas estações na rede ou uso intensivo do Dashboard/relatórios.

Componente	Recomendado	Observação
Processador (CPU)	Intel Core i5 (10ª geração+) / AMD Ryzen 5 5000 series+	Melhora tempo de exportação de relatórios grandes (XLSX/CSV)
Memória RAM	16 GB	Confortável com MySQL, Electron e visualizador 3D abertos simultaneamente
Armazenamento	SSD NVMe, 50 GB+ livres	Acomoda anos de manuais em PDF, imagens de itens e arquivos 3D sem necessidade de limpeza recorrente
Servidor de banco de dados (se compartilhado)	Máquina dedicada com 8 GB RAM, SSD, MySQL 8.0 em Docker ou instalação nativa	Recomendado a partir de 3+ estações usando o mesmo banco simultaneamente
No-break (UPS)	600VA ou superior, na máquina de estoque e no servidor de banco	Mitiga corrupção de transação por queda de energia abrupta — barato e frequentemente ignorado

17.4 Sistemas Operacionais Suportados

Sistema Operacional	Suporte	Observação
Windows 11 (64-bit)	Totalmente suportado — plataforma primária	Alvo principal do instalador NSIS (Seção 21)
Windows 10 (64-bit, versão 1909+)	Totalmente suportado	Ainda comum em chão de fábrica; testado no pipeline de CI
Windows Server 2019/2022	Suportado apenas como host do MySQL (servidor de banco)	Não recomendado para rodar a interface Electron diretamente
Linux (Ubuntu 22.04 LTS+)	Suportado experimentalmente	Electron é multiplataforma por natureza; requer build adicional (<code>electron-builder</code> com target <code>AppImage</code> / <code>deb</code>), não incluído no roadmap inicial da Seção 22
macOS	Não suportado nesta versão	Fora de escopo — ambiente de almoxarifado industrial tipicamente usa Windows
Windows 8.1 ou anterior	Não suportado	Fora do ciclo de suporte da Microsoft; Electron/Chromium moderno não garante compatibilidade

17.5 Requisitos de Rede e Periféricos

Item	Requisito
Conectividade com o banco	Se o MySQL estiver em servidor de rede: latência < 5ms e rede local estável (Ethernet preferível a Wi-Fi para a estação de estoque)
Acesso à internet	Não obrigatório para operação — recomenda-se, inclusive, isolar a máquina de estoque da internet pública por segurança (Seção 4.6)
Scanner de código de barras/QR	USB, modo HID "keyboard wedge" (Seção 13.2) — nenhum requisito adicional de driver
Impressora de etiquetas	Térmica com suporte a ZPL (Seção 14.4) ou impressora comum com suporte a impressão de PDF em folha adesiva (Seção 14.5)
Leitor de crachá (opcional)	Caso a empresa utilize leitura de crachá por RFID/magnético para login, o dispositivo deve também operar em modo HID, seguindo o mesmo princípio do scanner de item

18. Auditoria, Logs e Compliance

Mesmo sendo um sistema interno sem dados de clientes externos, ele contém dados pessoais de colaboradores (Seção 4) e está sob o guarda-chuva da LGPD no que se refere a dados de funcionários.

18.1 Ações que Geram Log de Auditoria Obrigatório

Ação	Código do log	Dados registrados
Login bem-sucedido	LOGIN	usuário, IP da máquina, data/hora
Login falho	LOGIN_FALHO	crachá tentado, IP, data/hora, contagem de tentativas
Reset de senha	RESET_SENHA	quem executou, beneficiário, data/hora
Criação de item	CRIAR_ITEM	usuário, payload completo em dados_novos
Edição de item	EDITAR_ITEM	usuário, dados_anteriores e dados_novos (diff completo)
Solicitação de remoção	SOLICITAR_REMOCAO	usuário, item, motivo
Aprovação/rejeição de remoção	APROVAR_REMOCAO / REJEITAR_REMOCAO	aprovador, justificativa (se rejeitado)
Registro de movimentação	MOVIMENTACAO_ESTOQUE	usuário, item, tipo, quantidade
Exportação de relatório	EXPORTAR_RELATORIO	usuário, tipo de relatório, filtros aplicados
Alteração de configuração do sistema	ALTERAR_CONFIG	ADM responsável, campo alterado, valor anterior/novo

Imutabilidade: a tabela `logs_auditoria` deve ser acessada pela aplicação com um usuário de banco de dados que possui exclusivamente os privilégios `INSERT`, `SELECT` — nunca `UPDATE` ou `DELETE`. Log que pode ser apagado não é auditoria, é apenas uma sugestão de que algo aconteceu.

```
-- Exemplo de criacao de usuario de aplicacao com privilegios minimos
CREATE USER 'app_estoque'@'localhost' IDENTIFIED BY 'senha-forte-aqui';
GRANT SELECT, INSERT, UPDATE, DELETE ON estoque_interno.* TO 'app_estoque'@'localhost';
REVOKE UPDATE, DELETE ON estoque_interno.logs_auditoria FROM 'app_estoque'@'localhost';
FLUSH PRIVILEGES;
```

18.2 Retenção de Logs

Mínimo de 5 anos, conforme Seção 4.4. Recomenda-se rotina de arquivamento (não exclusão) de logs com mais de 2 anos para uma tabela de histórico separada, mantendo a tabela operacional performática sem perder o registro histórico.

19. Backup e Recuperação de Desastres

19.1 Objetivos de Recuperação

Métrica	Meta
RPO (Recovery Point Objective) — perda de dados máxima aceitável	24 horas (backup diário)
RTO (Recovery Time Objective) — tempo máximo de indisponibilidade	4 horas úteis

19.2 Estratégia de Backup

- **Backup lógico diário** via `mysqldump` agendado (Task Scheduler do Windows), com retenção de 30 dias em rotação;
- **Backup dos arquivos** (`storage/imagens-itens` , `storage/manuais` , `storage/modelos-3d`) via cópia incremental para pasta de rede/NAS da empresa;
- **Teste de restauração trimestral obrigatório** — backup nunca testado em restauração é apenas uma esperança, não uma garantia real de recuperação;
- **Backup fora do local físico (offsite)** mensal, mitigando risco de sinistro (incêndio, roubo) que afete simultaneamente o servidor e o backup local.

```
#!/bin/bash
DATA=$(date +%Y%m%d_%H%M)
mysqldump -u backup_user -p estoque_interno > /backups/estoque_$DATA.sql
find /backups -name "estoque_*.sql" -mtime +30 -delete
```

19.3 Plano de Recuperação de Desastres — Passo a Passo

1. Identificar a última cópia de backup íntegra disponível (verificar checksum/tamanho do arquivo);
2. Provisionar/restaurar a instância MySQL em ambiente limpo;
3. Restaurar o dump via `mysql -u root -p estoque_interno < backup.sql` ;
4. Restaurar os arquivos de `storage/` a partir do backup incremental mais recente;
5. Validar integridade: contagem de registros por tabela, comparação com o snapshot do dashboard antes do incidente;
6. Comunicar aos usuários o horário de retomada e possível janela de dados perdidos (conforme RPO).

20. Estrutura de Pastas e Padrões de Código

20.1 Estrutura de Pastas

```
estoque-interno/
├─ electron/
│   ├── main.ts
│   ├── preload.ts
│   ├── database/ (schema.prisma, migrations/)
│   ├── services/ (auth, itens, movimentacoes, relatorios, etiquetas, dashboard)
│   ├── ipc/ (auth.ipc.ts, itens.ipc.ts, usuarios.ipc.ts, relatorios.ipc.ts)
│   ├── security/ (hashing.ts, rbac.ts, session.ts)
│   └─ printing/ (etiqueta-zpl.ts)
├─ src/ (Renderer - React)
│   ├── pages/ (Login, Dashboard, Estoque, EntradasSaidas, Usuarios, Perfil, Sobre)
│   ├── components/
│   ├── hooks/
│   ├── store/ (Zustand)
│   └─ styles/
├─ storage/ (gerado em runtime, fora do bundle)
│   ├── imagens-itens/
│   ├── manuais/
│   ├── modelos-3d/
│   └─ logs/
├─ docs/ (esta documentação + manuais internos)
├─ e2e/ (testes Playwright)
├─ electron-builder.yml
├─ package.json
└─ tsconfig.json
```

20.2 Padrões de Código

Ferramenta	Finalidade
ESLint (config Airbnb + TypeScript)	Padronização de estilo e prevenção de erros comuns
Prettier	Formatação automática consistente
Husky + lint-staged	Bloqueio de commit com código fora do padrão
Conventional Commits	Padrão de mensagens de commit (feat: , fix: , chore:) para changelog automático

20.3 Checklist de Revisão de Código (Pull Request)

- Toda nova rota/canal IPC possui verificação de RBAC explícita?
- Toda entrada de usuário é validada via schema Zod?
- Alterações de schema do banco vêm acompanhadas de migration versionada?
- Ações sensíveis (criar, editar, remover, resetar senha) geram log de auditoria?
- Testes unitários/E2E cobrem o novo comportamento?
- Nenhuma credencial ou segredo foi commitado (verificação automática via `git-secrets` ou similar)?

21. CI/CD e Empacotamento

```
# electron-builder.yml
appId: com.empresa.estoqueinterno
productName: "Estoque Interno"
files:
  - dist/**/*
  - electron/**/*
win:
  target: nsis
  icon: build/icon.ico
nsis:
  oneClick: false
  allowToChangeInstallationDirectory: true
publish: null # distribuicao interna, sem auto-update publico
```

21.1 Pipeline de CI (exemplo GitHub Actions)

```
name: build-and-test
on: [push, pull_request]
jobs:
  test:
    runs-on: windows-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm ci
      - run: npm run lint
      - run: npm run test:unit
      - run: npm run build
      - run: npm run test:e2e
  package:
    needs: test
    runs-on: windows-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm run build && npx electron-builder
```

21.2 Distribuição Interna

Como o uso é 100% interno, a distribuição do instalador é feita via pasta de rede compartilhada ou repositório interno (Nexus/Artifactory), sem necessidade de certificado de assinatura de código pago (EV) — ainda que seja recomendável assinar com certificado interno da empresa para evitar alertas do SmartScreen do Windows ao instalar em máquinas novas.

22. Plano de Implementação (Roadmap)

Fase	Entregas	Estimativa
Fase 1 — Fundação	Setup Electron+React+TS, schema MySQL/Prisma, autenticação, RBAC básico, Termos de Uso	3 semanas
Fase 2 — Estoque Core	CRUD de itens, prateleiras, máquinas, geração de etiquetas	3 semanas
Fase 3 — Movimentações	Entradas/saídas, integração com scanner, aprovação de remoção	2 semanas
Fase 4 — Relatórios & Dashboard	KPIs, gráficos, exportação CSV/XLSX, aplicação do Sistema de Design	2 semanas
Fase 5 — Extras	Visualizador 3D, gestão de manuais, aba Sobre, política de dados (LGPD)	2 semanas
Fase 6 — Hardening	Testes E2E, revisão de segurança (checklist Seção 9.5), empacotamento, treinamento de usuários	2 semanas

Total estimado: ~14 semanas para uma equipe de 1-2 desenvolvedores full-stack em dedicação integral.

23. Treinamento e Onboarding de Usuários

23.1 Trilha de Treinamento por Perfil

Perfil	Conteúdo do treinamento	Duração sugerida
Operador	Login, leitura via scanner, registro de saída, consulta de estoque	30 minutos
Técnico	Tudo do Operador + cadastro de item, cadastro de máquina, solicitação de remoção	1 hora
Analista	Tudo do Técnico + aprovação de remoção, relatórios, Dashboard, exportação	1h30
GL	Tudo do Analista + gestão de usuários, reset de senha, notificações de aprovação	2 horas
ADM	Tudo do GL + configurações do sistema, backup, gestão de cargos/departamentos	3 horas

23.2 Checklist de Onboarding de Novo Colaborador

1. Cadastro do usuário no sistema pelo GL/ADM (crachá, nome, departamento, cargo, líder);
2. Entrega presencial da senha temporária;
3. Primeiro login: aceite obrigatório dos Termos de Uso (Seção 3) e troca de senha;
4. Treinamento prático conforme a trilha do perfil correspondente (tabela acima);
5. Acompanhamento das primeiras 3 movimentações reais por um colega mais experiente ("shadow" operacional);
6. Registro de conclusão do onboarding no sistema de RH (fora do escopo deste software, mas recomendado como boa prática).

23.3 Material de Apoio Recomendado

- Cartão de referência rápida plastificado, fixado próximo ao terminal, com o fluxo resumido de retirada via scanner (Figura da Seção 12.2);
- Vídeo curto (3–5 min) gravado internamente demonstrando o fluxo completo de cadastro e retirada;
- FAQ dentro da própria aba "Sobre" (Seção 11.2), cobrindo dúvidas recorrentes de uso.

24. Glossário de Termos

Termo	Definição
ACID	Atomicidade, Consistência, Isolamento e Durabilidade — propriedades garantidas por transações em bancos relacionais como o MySQL/InnoDB
Ativo Fixo	Item de estoque que não é consumido no uso (ex: máquina, ferramenta durável), sujeito a controle patrimonial de longo prazo
Item de Consumo	Item que é utilizado/gasto no processo produtivo (ex: parafuso, luva, óleo lubrificante)
RBAC	Role-Based Access Control — modelo de controle de acesso baseado em cargos/papéis
GL	Group Leader — líder de grupo, cargo com permissões amplas de gestão de usuários
ADM	Administrador — cargo com acesso total ao sistema
Argon2id	Algoritmo de hash de senha recomendado pela OWASP, vencedor da Password Hashing Competition
IPC	Inter-Process Communication — mecanismo de comunicação entre o Main Process e o Renderer Process no Electron
HID	Human Interface Device — classe de dispositivos USB (teclado, mouse, scanner em modo wedge) reconhecidos nativamente pelo SO
LGPD	Lei Geral de Proteção de Dados (Lei nº 13.709/2018), legislação brasileira de proteção de dados pessoais
RPO / RTO	Recovery Point Objective / Recovery Time Objective — métricas de perda de dados e tempo de recuperação aceitáveis em um desastre
ZPL	Zebra Programming Language — linguagem de comando usada por impressoras térmicas de etiqueta
STRIDE	Modelo de ameaças de segurança (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege)
OWASP ASVS	Application Security Verification Standard — checklist de requisitos de segurança de aplicações mantido pela OWASP

25. Anexos

Anexo A — Exemplo de Registro de Log de Auditoria

```
{
  "id": 88231,
  "usuario_id": 14,
  "acao": "EDITAR_ITEM",
  "tabela_afetada": "itens",
  "registro_id": 1042,
  "dados_anteriores": {
    "quantidade_minima": 10,
    "preco_unitario": 89.90
  },
  "dados_novos": {
    "quantidade_minima": 15,
    "preco_unitario": 94.50
  },
  "ip_maquina": "192.168.1.42",
  "data_hora": "2026-07-30T14:22:03Z"
}
```

Anexo B — Exemplo de Payload Completo de Cadastro de Item

```
{
  "nome": "Rolamento Blindado 6205-2RS",
  "modelo": "6205-2RS",
  "marca": "SKF",
  "precoUnitario": 34.90,
  "tipo": "CONSUMO",
  "prateleiraId": 12,
  "maquinaEquipamentoId": 7,
  "descricao": "Rolamento blindado, uso em motores de esteira transportadora.",
  "quantidadeInicial": 40,
  "quantidadeMinima": 10,
  "manuais": [
    { "tipo": "LINK", "titulo": "Catalogo SKF", "url": "https://www.skf.com/..." }
  ]
}
```

Anexo C — Exemplo de Resposta de Dashboard (canal `dashboard:kpis`)

```
{
  "valorTotalEstoque": 482930.00,
  "itensCadastrados": 3482,
  "itensAbaixoMinimo": 17,
  "retiradasNoMes": 612,
  "topItensRetirados": [
    { "item": "Parafuso Sextavado M6", "quantidade": 890 },
    { "item": "Luva Nitrílica", "quantidade": 640 }
  ]
}
```

Anexo D — Checklist Rápido de Instalação em Nova Máquina

1. Instalar MySQL 8.0+ (ou configurar acesso ao servidor de rede interna);
2. Executar o script DDL completo (Seção 8.2) para criar o banco;
3. Executar o instalador `EstoqueInterno-Setup.exe` ;
4. Configurar a string de conexão com o banco no primeiro assistente de instalação;
5. Cadastrar o primeiro usuário ADM manualmente via script de seed (fora da interface, por segurança);
6. Validar impressora de etiquetas conectada e configurada como padrão;
7. Testar leitura de scanner com uma etiqueta de exemplo.

26. Guia de Desenvolvimento Passo a Passo (Tutorial Completo de Implementação)

Este anexo é o **tutorial operacional de construção do GWM ASTHA**, escrito para ser seguido sequencialmente por um time de desenvolvimento — do primeiro `git init` até o go-live em produção. Cada passo apresenta: objetivo, procedimento técnico detalhado, **Definição de Pronto** (Definition of Done), testes padrão de indústria a serem escritos/executados, e testes específicos de segurança e dados. Nenhum passo deve ser considerado concluído sem que sua Definição de Pronto seja satisfeita — este é o mecanismo de controle de qualidade de todo o projeto.

26.1 Modelo de Qualidade Esperado (Referência Geral — ISO/IEC 25010)

Todo o desenvolvimento segue as características de qualidade de software da norma **ISO/IEC 25010**, adaptadas ao contexto do GWM ASTHA. Cada passo do tutorial referencia explicitamente quais destas características está reforçando:

Característica	Meta aplicada ao GWM ASTHA
Adequação Funcional	100% dos requisitos funcionais das Seções 2 e 11 implementados e cobertos por teste de aceitação
Eficiência de Desempenho	Busca de item < 200ms; abertura do app < 5s; exportação de 10 mil linhas < 8s
Compatibilidade	Windows 10/11 64-bit; MySQL 8.0+; impressoras ZPL e genéricas
Usabilidade	Retirada via scanner em < 10s; WCAG AA nas telas principais
Confiabilidade	Zero perda de dados em queda de energia (transações ACID); MTBF alvo > 720h de uso contínuo
Segurança	Conformidade com o checklist OWASP ASVS da Seção 9.5; zero senha em texto plano; RBAC revalidado em 100% dos canais IPC
Manutenibilidade	Cobertura de testes ≥ 80% em <code>services/</code> ; complexidade ciclomática máxima 10 por função (ESLint)
Portabilidade	Instalação em nova máquina em até 30 minutos via checklist do Anexo D

Como usar este guia: cada Passo abaixo corresponde a uma ou mais Sprints do Roadmap (Seção 22). Um Passo só avança para o próximo depois que sua tabela de "Definição de Pronto" estiver 100% marcada e os testes de segurança listados tiverem sido executados com resultado aprovado, registrado em ata de revisão de sprint.

PASSO 1 — Preparação do Ambiente e Scaffolding do Projeto

Objetivo: ter um projeto Electron+React+TypeScript rodando localmente, com lint, formatação e Git configurados, antes de qualquer linha de regra de negócio.

- Instalar Node.js LTS (20.x), Git e MySQL Workbench na máquina de desenvolvimento;
- Criar o repositório: `git init gwm-astha-estoque && cd gwm-astha-estoque` ;
- Inicializar o projeto com template Electron + Vite + React + TS: `npm create vite@latest . -- --template react-ts` , seguido da integração do Electron via `electron-vite` ou `vite-plugin-electron` ;
- Configurar ESLint (config Airbnb + TypeScript) e Prettier, com `.eslintrc` e `.prettierrc` versionados;

5. Configurar Husky + lint-staged para bloquear commits fora do padrão;
6. Criar a estrutura de pastas completa conforme Seção 20.1;
7. Configurar Conventional Commits (`commitlint`) e o primeiro pipeline de CI (Seção 21.1) rodando apenas `lint` e `build` nesta fase.

Definição de Pronto — Passo 1	Status esperado
Aplicação Electron abre uma janela em branco sem erros no console	Obrigatório
<code>npm run lint</code> executa sem erros na estrutura inicial	Obrigatório
Pipeline de CI verde no primeiro commit (branch principal protegida)	Obrigatório
README.md inicial com instruções de setup para outro desenvolvedor	Obrigatório

```
terminal - gwm-astha-estoque

$ git init gwm-astha-estoque
$ cd gwm-astha-estoque
$ npm create vite@latest . -- --template react-ts
✓ Project scaffolded successfully.
$ npm install electron electron-builder -D
$ npm install prisma @prisma/client argon2 zod qrcode exceljs
added 187 packages in 14s
$ npx husky init && npx commitlint --init
$ npm run lint
✓ No ESLint errors found.
$ npm run dev
VITE v5.2.0 ready in 341 ms
→ Local: http://localhost:5173/
→ electron-app started (main.ts loaded)
✓ Janela GWM ASTHA aberta sem erros no console.

estrutura de pastas gerada:
gwm-astha-estoque/
├─ electron/ (main.ts, preload.ts, database/, services/, ipc/, security/)
├─ src/ (pages/, components/, hooks/, store/)
├─ storage/ (imagens-itens/, manuais/, modelos-3d/, logs/)
├─ e2e/
├─ package.json
└─ tsconfig.json
```

Figura 7 — Sessão de terminal ilustrando o scaffolding inicial do projeto (saída ilustrativa).

1.1 Arquivo `package.json` de Referência

```
{
  "name": "gwm-astha-estoque",
  "version": "0.1.0",
  "private": true,
  "main": "dist-electron/main.js",
  "scripts": {
    "dev": "vite",
    "build": "tsc && vite build && electron-builder --dir",
    "lint": "eslint . --ext .ts,.tsx",
    "test:unit": "vitest run --coverage",
    "test:e2e": "playwright test",
    "prisma:migrate": "prisma migrate dev",
    "prisma:seed": "tsx prisma/seed.ts",
    "package": "electron-builder"
  },
  "dependencies": {
    "react": "^18.3.0",
    "react-dom": "^18.3.0",
    "@prisma/client": "^5.16.0",
    "argon2": "^0.31.2",
  }
}
```

```

    "zod": "^3.23.8",
    "qrcode": "^1.5.3",
    "exceljs": "^4.4.0"
  },
  "devDependencies": {
    "electron": "^31.0.0",
    "electron-builder": "^24.13.3",
    "vite": "^5.2.0",
    "typescript": "^5.4.5",
    "vitest": "^1.6.0",
    "@playwright/test": "^1.44.0",
    "eslint": "^8.57.0",
    "prettier": "^3.3.0",
    "husky": "^9.0.11"
  }
}

```

1.2 Esqueleto Inicial do `electron/main.ts`

```

import { app, BrowserWindow } from 'electron';
import path from 'node:path';

function createWindow() {
  const win = new BrowserWindow({
    width: 1440,
    height: 900,
    webPreferences: {
      preload: path.join(__dirname, 'preload.js'),
      contextIsolation: true, // OBRIGATORIO - Secao 9.4
      nodeIntegration: false, // OBRIGATORIO - Secao 9.4
      sandbox: true, // OBRIGATORIO - Secao 9.4
    },
  });

  if (process.env.NODE_ENV === 'development') {
    win.loadURL('http://localhost:5173');
    win.webContents.openDevTools();
  } else {
    win.loadFile(path.join(__dirname, '../dist/index.html'));
    // DevTools NUNCA habilitado em producao - ver Passo 17
  }
}

app.whenReady().then(createWindow);

```

1.3 Problemas Esperados e Correções — Passo 1

Problema	Causa provável	Correção
Janela do Electron abre em branco, sem erro no terminal	URL do Vite (<code>localhost:5173</code>) ainda não estava de pé quando o Electron tentou carregar	Adicionar espera/retry no <code>main.ts</code> ou usar <code>wait-on http://localhost:5173</code> antes de iniciar o Electron no script <code>dev</code>
Erro <code>Cannot find module 'electron'</code> ao rodar build	Dependência instalada apenas como <code>devDependency</code> em ambiente de CI que faz <code>npm ci --omit=dev</code>	Garantir que o pipeline de build (não o de teste) instale todas as dependências, incluindo dev
<code>npm run lint</code> falha com centenas de erros no primeiro commit	Config ESLint aplicada sobre arquivos de template gerados automaticamente (ex: <code>vite-env.d.ts</code>)	Adicionar <code>.eslintignore</code> para arquivos gerados e de terceiros

Husky não bloqueia commit fora do padrão	Hook não foi instalado após clonar o repositório em outra máquina	Adicionar script <code>"prepare": "husky install"</code> no <code>package.json</code> , executado automaticamente no <code>npm install</code>
--	---	---

Testes padrão de indústria: smoke test manual de abertura da aplicação; teste de CI validando que o build de produção (`electron-builder --dir`) gera artefato sem erro.

Testes de segurança e dados: validar que `.gitignore` exclui `node_modules`, arquivos `.env` e qualquer credencial; rodar `git-secrets --scan` no primeiro commit para garantir que nenhum segredo foi versionado por engano.

PASSO 2 — Banco de Dados: Provisionamento e Migrations

Objetivo: ter o schema completo da Seção 8.2 versionado via Prisma Migrate, com um banco de desenvolvimento populado por dados de teste (seed).

1. Provisionar instância MySQL 8.0+ local (ou container Docker de desenvolvimento);
2. Instalar Prisma: `npm install prisma @prisma/client` e `npx prisma init --datasource-provider mysql`;
3. Traduzir o DDL da Seção 8.2 para `schema.prisma`, tabela por tabela, revisando tipos e relações;
4. Gerar a primeira migration: `npx prisma migrate dev --name init`;
5. Criar script de seed (`prisma/seed.ts`) com cargos padrão, um usuário ADM inicial e dados fictícios de itens/prateleiras para desenvolvimento;
6. Configurar usuário de banco de dados de aplicação com privilégios restritos (sem UPDATE/DELETE em `logs_auditoria`, conforme Seção 18.1).

Definição de Pronto — Passo 2	Status esperado
Todas as 16 tabelas da Seção 8.2 criadas sem erro via migration	Obrigatório
Seed executa e popula banco de desenvolvimento em < 10s	Obrigatório
Usuário de aplicação sem privilégio de UPDATE/DELETE em <code>logs_auditoria</code> (validado via <code>SHOW GRANTS</code>)	Obrigatório
Diagrama ER gerado automaticamente (<code>prisma-erd-generator</code>) confere com a Figura 3	Recomendado

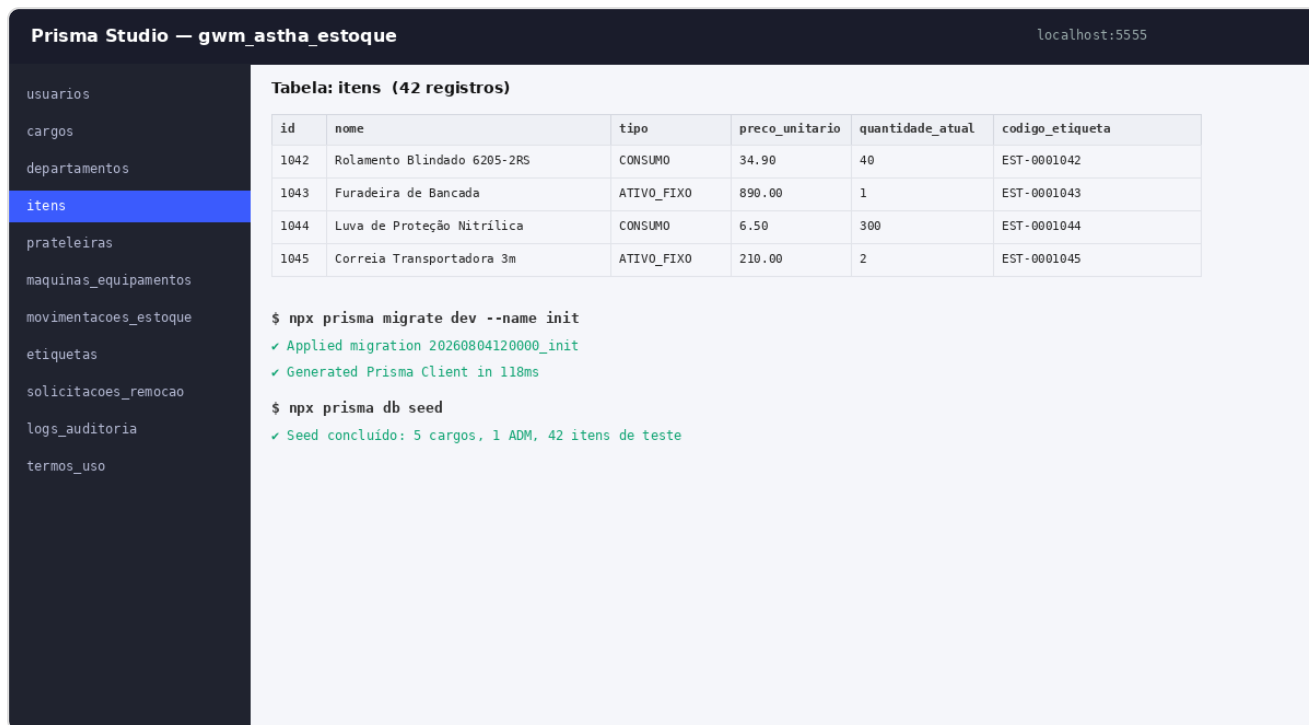


Figura 8 — Prisma Studio (ferramenta oficial do Prisma) exibindo a tabela **itens** após seed, com o resultado da migration no terminal (mockup ilustrativo).

2.1 Trecho de `schema.prisma` — Modelo **Item**

```

model Item {
  id          Int           @id @default(autoincrement())
  nome        String        @db.VarChar(150)
  modelo      String?       @db.VarChar(100)
  marca       String?       @db.VarChar(100)
  precoUnitario Decimal     @map("preco_unitario") @db.Decimal(12,2)
  imagem      String?       @db.VarChar(255)
  tipo        TipoItem
  maquinaEquipamentoId Int? @map("maquina_equipamento_id")
  prateleiraId Int          @map("prateleira_id")
  descricao   String?       @db.Text
  arquivo3d   String?       @map("arquivo_3d") @db.VarChar(255)
  quantidadeAtual Int       @default(0) @map("quantidade_atual")
  quantidadeMinima Int      @default(0) @map("quantidade_minima")
  codigoEtiqueta String     @unique @map("codigo_etiqueta") @db.VarChar(30)
  ativo       Boolean       @default(true)
  criadoPor   Int           @map("criado_por")

  prateleira      Prateleira      @relation(fields: [prateleiraId], references: [id])
  maquinaEquipamento MaquinaEquipamento? @relation(fields: [maquinaEquipamentoId], references: [id])
  movimentacoes   MovimentacaoEstoque[]

  @@map("itens")
  @@index([tipo])
  @@index([prateleiraId])
}

enum TipoItem {
  CONSUMO
  ATIVO_FIXO
}

```

2.2 Script de Seed — Trecho de `prisma/seed.ts`

```
import { PrismaClient } from '@prisma/client';
import argon2 from 'argon2';

const prisma = new PrismaClient();

async function main() {
  await prisma.cargo.createMany({
    data: [
      { nome: 'Operador', nivelAcesso: 1 },
      { nome: 'Tecnico', nivelAcesso: 2 },
      { nome: 'Analista', nivelAcesso: 3 },
      { nome: 'GL', nivelAcesso: 4 },
      { nome: 'ADM', nivelAcesso: 5 },
    ],
  });

  const senhaHash = await argon2.hash('TrocarNoPrimeiroLogin!23', { type: argon2.argon2id });
  await prisma.usuario.create({
    data: {
      cracha: '00001',
      nomeCompleto: 'Administrador Inicial',
      nomeUsuario: 'admin',
      senhaHash,
      departamentoId: 1,
      cargoId: 5, // ADM
      deveTrocarSenha: true,
    },
  });
}

main().finally(() => prisma.$disconnect());
```

2.3 Problemas Esperados e Correções — Passo 2

Problema	Causa provável	Correção
Erro <code>P1001: Can't reach database server</code> ao migrar	MySQL não está rodando, ou <code>DATABASE_URL</code> no <code>.env</code> aponta para host/porta errados	Validar <code>mysqladmin ping -h localhost</code> antes de migrar; confirmar porta 3306 liberada no firewall local
Erro <code>P3009: migrate found failed migration</code>	Uma migration anterior foi interrompida no meio (ex: queda de energia durante <code>migrate dev</code>)	Rodar <code>npx prisma migrate resolve --rolled-back <nome></code> e reaplicar; em desenvolvimento, <code>prisma migrate reset</code> é aceitável
Seed falha com <code>Unique constraint failed on cracha</code>	Seed executado mais de uma vez sem limpar o banco	Tornar o seed idempotente com <code>upsert</code> em vez de <code>create</code> , ou documentar que o seed é "somente primeira execução"
<code>FULLTEXT INDEX</code> não é criado pela migration do Prisma	Prisma ORM (na versão usada) não gera índices FULLTEXT nativamente via schema declarativo	Adicionar o índice manualmente em uma migration SQL customizada (<code>prisma migrate dev --create-only</code> + editar o SQL gerado antes de aplicar)

Testes padrão de indústria: teste de integração que roda a migration em banco limpo e verifica a existência de cada tabela/coluna esperada; teste de "migration rollback" garantindo que `prisma migrate reset` funciona sem deixar resíduo.

Testes de segurança e dados: tentativa manual de `DELETE FROM logs_auditoria` com o usuário de aplicação deve falhar com erro de permissão (teste negativo obrigatório); validar que a string de conexão nunca é commitada em texto plano (uso de `.env` + `dotenv`, ignorado pelo Git).

PASSO 3 — Camada de Segurança Base (Hashing, RBAC, Sessão)

Objetivo: implementar a espinha dorsal de segurança (Seção 9) antes de qualquer tela funcional, já que toda funcionalidade subsequente depende dela.

1. Implementar `security/hashing.ts` com Argon2id (parâmetros da Seção 9.1);
2. Implementar `security/rbac.ts` com a tabela de permissões por canal (Seção 10.2);
3. Implementar `security/session.ts` — geração/validação de JWT interno, armazenamento da chave via `keytar` ;
4. Configurar `contextIsolation: true` , `nodeIntegration: false` , `sandbox: true` na janela principal do Electron (Seção 9.4);
5. Criar o `preload.ts` inicial expondo apenas um placeholder de API via `contextBridge` ;
6. Escrever testes unitários cobrindo 100% das funções desta camada antes de seguir para o Passo 4 (regra inegociável — segurança sem teste não é segurança, é aposta).

Definição de Pronto — Passo 3	Status esperado
Hash e verificação de senha funcionando com Argon2id	Obrigatório
<code>verificarPermissao()</code> bloqueia corretamente todos os níveis testados (1 a 5)	Obrigatório
JWT expira corretamente após 8h em teste controlado (mock de tempo)	Obrigatório
Cobertura de testes unitários $\geq 95\%$ neste módulo	Obrigatório

3.1 Implementação de `security/session.ts`

```
import jwt from 'jsonwebtoken';
import keytar from 'keytar';

const SERVICE = 'gwm-astha-estoque';
const ACCOUNT = 'jwt-signing-key';

async function obterChave(): Promise<string> {
  let chave = await keytar.getPassword(SERVICE, ACCOUNT);
  if (!chave) {
    chave = crypto.randomUUID() + crypto.randomUUID();
    await keytar.setPassword(SERVICE, ACCOUNT, chave);
  }
  return chave;
}

export async function gerarSessao(usuario: UsuarioAutenticado) {
  const chave = await obterChave();
  return jwt.sign(
    { usuarioId: usuario.id, cargo: usuario.cargo, nivelAcesso: usuario.nivelAcesso },
    chave,
    { expiresIn: '8h' },
  );
}

export async function validarSessao(token: string) {
  const chave = await obterChave();
  try {
    return jwt.verify(token, chave) as SessaoUsuario;
  } catch {
    throw new ErroSessaoInvalida('Sessao expirada ou token invalido');
  }
}
```

3.2 Teste Unitário de Referência

```
describe('rbac.verificarPermissao', () => {
  it.each([
    ['itens:criar', 1, true], // Operador tentando criar item -> deve bloquear
    ['itens:criar', 2, false], // Tecnico criando item -> deve permitir
    ['usuarios:resetar-senha', 3, true], // Analista resetando senha -> deve bloquear
    ['usuarios:resetar-senha', 4, false], // GL resetando senha -> deve permitir
  ])('canal %s com nivel %i deve lancar erro = %s', (canal, nivel, deveLancar) => {
    const sessao = { nivelAcesso: nivel } as SessaoUsuario;
    if (deveLancar) {
      expect(() => verificarPermissao(canal, sessao)).toThrow(ErroPermissaoNegada);
    } else {
      expect(() => verificarPermissao(canal, sessao)).not.toThrow();
    }
  });
});
```

3.3 Problemas Esperados e Correções — Passo 3

Problema	Causa provável	Correção
Erro <code>keytar</code> não compila no Windows (<code>node-gyp</code>)	Ferramentas de build C++ do Windows não instaladas	Instalar <code>windows-build-tools</code> ou usar a versão pré-compilada do keytar compatível com a versão do Electron (<code>electron-rebuild</code>)
Hash de senha demora mais de 1s por chamada	Parâmetros de <code>memoryCost</code> / <code>timeCost</code> configurados acima do necessário para o hardware da Seção 17.2	Recalibrar para ~19MB/2 iterações (padrão OWASP) — o objetivo é ser lento o suficiente para dificultar força bruta, não lento a ponto de frustrar o usuário
Token JWT válido em uma sessão passa a falhar após reiniciar o app	Chave de assinatura gerada em memória, não persistida via keytar corretamente	Garantir que <code>obterChave()</code> sempre leia do cofre do SO antes de gerar uma nova chave aleatória

Testes padrão de indústria: testes unitários (Vitest) para cada função de `hashing.ts`, `rbac.ts` e `session.ts`, incluindo casos de borda (senha vazia, nível de acesso inexistente, token expirado, token adulterado).

Testes de segurança e dados: teste negativo garantindo que dois hashes do mesmo valor de senha nunca são idênticos (salt automático do Argon2id); teste de força bruta simulada confirmando tempo de verificação $\geq$ 100ms por tentativa (mitigação de timing attack); teste garantindo que um JWT assinado com chave diferente é sempre rejeitado.

PASSO 4 — Módulo de Autenticação e Termos de Uso

Objetivo: tela de login funcional, ponta a ponta, com bloqueio de conta e aceite obrigatório dos Termos de Uso (Seção 3).

1. Implementar canal IPC `auth:login`, validando payload via Zod e consultando o Prisma;
2. Implementar contagem de tentativas falhas e bloqueio temporário (Seção 9.2);
3. Construir a tela de Login em React seguindo o Sistema de Design da Seção 7 (gradiente + cartão glass centralizado);
4. Implementar a tela bloqueante de aceite de Termos de Uso, consultando `termos_uso` / `aceites_termos` ;
5. Implementar troca de senha obrigatória quando `deve_trocar_senha = true` .

Definição de Pronto — Passo 4

	Status esperado
Login válido redireciona à tela principal; login inválido exibe erro genérico (sem revelar se o crachá existe)	Obrigatório
5ª tentativa incorreta bloqueia a conta por 15 minutos	Obrigatório
Usuário não pode acessar nenhuma outra tela sem aceitar os Termos vigentes	Obrigatório
Tela segue os tokens de design da Seção 7.1 (validação visual por QA)	Recomendado

4.1 Componente React — `LoginPage.tsx` (trecho)

```
export function LoginPage() {
  const [cracha, setCracha] = useState('');
  const [senha, setSenha] = useState('');
  const [erro, setErro] = useState<string | null>(null);
  const [carregando, setCarregando] = useState(false);

  async function handleLogin(e: FormEvent) {
    e.preventDefault();
    setCarregando(true);
    setErro(null);
    try {
      const resultado = await window.api.auth.login(cracha, senha);
      if (resultado.deveTrocarSenha) navigate('/trocar-senha');
      else navigate('/dashboard');
    } catch (err) {
      // Mensagem SEMPRE generica - nunca revelar se o cracha existe
      setErro('Crachá ou senha inválidos.');
```

4.2 Problemas Esperados e Correções — Passo 4

Problema	Causa provável	Correção
Usuário bloqueado não recebe nenhuma	Mensagem de erro genérica demais, prejudicando usabilidade	Exibir "Conta temporariamente bloqueada. Tente novamente em X minutos." — isso não compromete

mensagem clara sobre o tempo restante		segurança, pois o usuário já sabe que a conta existe (ele digitou a própria senha)
Tela de Termos de Uso aparece novamente após já ter sido aceita	Comparação de versão do termo feita por índice em vez de pela FK <code>termo_id</code> vigente	Buscar sempre o registro de <code>termos_uso</code> mais recente por <code>vigente_desde</code> e comparar o <code>id</code> exato
Fluxo de troca de senha obrigatória entra em loop infinito	Endpoint de troca de senha não está limpando a flag <code>deve_trocar_senha</code> após sucesso	Garantir que a transação de troca de senha sempre inclua <code>deve_trocar_senha = false</code> no mesmo UPDATE

Testes padrão de indústria: teste E2E (Playwright) do fluxo completo de login, bloqueio e desbloqueio; teste de integração do canal `auth:login` contra banco de teste.

Testes de segurança e dados: teste garantindo que a mensagem de erro de login é idêntica para "crachá inexistente" e "senha incorreta" (evita enumeração de usuários válidos); teste confirmando que a tentativa de acessar qualquer canal IPC sem sessão válida retorna erro de autorização; teste confirmando que o aceite de termos é gravado com data/hora e não pode ser duplicado (constraint `uk_usuario_termo`).

PASSO 5 — Módulo de Usuários (CRUD, Cargos, Reset de Senha)

Objetivo: permitir que GL/ADM gerenciem o quadro de colaboradores com segurança e auditoria completas.

1. Implementar canais `usuarios:listar`, `usuarios:criar`, `usuarios:resetar-senha`, `usuarios:desativar`;
2. Construir a tela "Gerenciar Usuários" com filtros por departamento/cargo/status;
3. Implementar a geração de senha temporária e a tela de exibição única (Seção 11.1.3);
4. Garantir soft-delete (nunca exclusão física) na desativação;
5. Conectar toda ação relevante ao log de auditoria (`CRIAR_USUARIO`, `RESET_SENHA`, `DESATIVAR_USUARIO`).

Definição de Pronto — Passo 5	Status esperado
Operador/Técnico não conseguem acessar a tela (RBAC validado na UI e no IPC)	Obrigatório
Senha temporária exibida apenas uma vez, nunca persistida em log em texto plano	Obrigatório
Usuário desativado não consegue mais logar, mas seu histórico permanece intacto	Obrigatório

5.1 Geração de Senha Temporária Segura

```
import crypto from 'node:crypto';

export function gerarSenhaTemporaria(): string {
  const alfabeto = 'ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz23456789!@#%$%';
  const bytes = crypto.randomBytes(12);
  return Array.from(bytes, (b) => alfabeto[b % alfabeto.length]).join('');
}

// Nota: exclui caracteres ambíguos (0/0, 1/l/I) para reduzir erro de digitacao
// ao repassar a senha temporaria presencialmente ao colaborador.
```

5.2 Problemas Esperados e Correções — Passo 5

Problema	Causa provável	Correção
Usuário desativado ainda aparece em seletores de "líder de grupo" de outros usuários	Query de listagem para seletores não filtra <code>ativo = true</code>	Aplicar filtro consistente em todos os seletores que referenciam <code>usuarios</code> , exceto nas telas de histórico/auditoria

Dois GLs resetam a senha do mesmo colaborador quase simultaneamente, gerando confusão sobre qual senha é válida

Ausência de invalidação da senha anterior/token de sessão ativo

Ao resetar senha, invalidar qualquer sessão JWT ativa daquele usuário (ex: incrementar um `tokenVersion` verificado na validação)

Testes padrão de indústria: teste de integração cobrindo criação, reset e desativação de usuário; teste E2E validando que um usuário Operador recebe erro 403/permissão negada ao tentar acessar a tela via URL/atalho direto.

Testes de segurança e dados: teste confirmando que a senha temporária nunca aparece em nenhum arquivo de log (grep automatizado nos logs de teste); teste confirmando que o campo `dados_novos` do log de auditoria de criação de usuário nunca contém a senha, apenas metadados.

PASSO 6 — Módulo de Estoque: Prateleiras, Máquinas e Itens

Objetivo: núcleo funcional do sistema — cadastro completo de itens conforme especificação da Seção 11.5.1.

1. Implementar CRUD de `prateleiras` e `maquinas_equipamentos`;
2. Implementar canal `itens:criar` com todos os campos e validações Zod da Seção 11.5.1;
3. Implementar upload e redimensionamento de imagem (thumbnail 512×512);
4. Implementar upload de arquivo 3D e manuais (armazenamento em `storage/`, nunca BLOB no MySQL);
5. Implementar busca full-text na tela de Estoque usando o índice `FULLTEXT` da Seção 8.2;
6. Disparar automaticamente, ao salvar: geração de `codigo_etiqueta`, registro em `etiquetas`, movimentação `ENTRADA` inicial e log de auditoria.

Definição de Pronto — Passo 6	Status esperado
Item cadastrado aparece imediatamente na busca e no dashboard	Obrigatório
Upload de arquivo maior que o limite (5MB imagem / 30MB 3D) é rejeitado com mensagem clara	Obrigatório
<code>codigo_etiqueta</code> gerado é sempre único (teste de concorrência com criação simultânea)	Obrigatório

6.1 Geração de Código Único com Proteção de Concorrência

```
export async function gerarCodigoEtiquetaUnico(tx: Prisma.TransactionClient): Promise<string> {
  // Usa uma tabela de contador dedicada com UPDATE atomico, em vez de
  // "SELECT MAX(id)+1", que sofre condicao de corrida sob carga concorrente.
  const contador = await tx.contadorEtiqueta.update({
    where: { id: 1 },
    data: { valorAtual: { increment: 1 } },
  });
  const numero = contador.valorAtual.toString().padStart(7, '0');
  return `EST-${numero}`;
}
```

6.2 Validação de Upload (Magic Bytes, não apenas extensão)

```
import { fileTypeFromBuffer } from 'file-type';

const TIPOS_IMAGEM_PERMITIDOS = ['image/jpeg', 'image/png', 'image/webp'];

export async function validarImagemUpload(buffer: Buffer) {
  const tipo = await fileTypeFromBuffer(buffer);
  if (!tipo || !TIPOS_IMAGEM_PERMITIDOS.includes(tipo.mime)) {
    throw new ErroUploadInvalido('Arquivo nao e uma imagem valida (verificacao por assinatura binaria).');
  }
  if (buffer.length > 5 * 1024 * 1024) {
```

```

    throw new ErroUploadInvalido('Imagem excede o limite de 5MB.');
```

6.3 Problemas Esperados e Correções — Passo 6

Problema	Causa provável	Correção
Dois cadastros simultâneos geram o mesmo <code>codigo_etiqueta</code>	Geração de código baseada em <code>COUNT(*)</code> ou <code>MAX(id)</code> , vulnerável a condição de corrida	Usar contador atômico dedicado (Seção 6.1 acima) ou <code>AUTO_INCREMENT</code> nativo do próprio banco combinado com o ID gerado
Busca full-text não retorna resultados esperados com termos parciais (ex: "parafus")	<code>MATCH()</code> <code>AGAINST()</code> em modo natural language não faz prefix-match por padrão	Usar modo booleano com wildcard: <code>MATCH(nome) AGAINST('parafus*' IN BOOLEAN MODE)</code>
Upload de arquivo 3D de 29MB trava a interface por alguns segundos	Processamento de upload rodando de forma síncrona no Main Process	Mover o processamento para um worker thread ou usar streaming de escrita em disco em vez de carregar o buffer inteiro em memória

Testes padrão de indústria: teste unitário do gerador de código único; teste de integração de todo o fluxo de cadastro (payload → banco → etiqueta → log); teste de carga simulando 50 cadastros simultâneos para validar ausência de colisão de código.

Testes de segurança e dados: teste de upload de arquivo malicioso (ex: executável renomeado como .jpg) deve ser rejeitado por validação de assinatura de arquivo (magic bytes), não apenas pela extensão; teste garantindo sanitização de nome de arquivo (prevenção de path traversal, ex: `../../etc/passwd`).

PASSO 7 — Geração de Etiquetas (QR Code) e Impressão

Objetivo: emitir etiquetas físicas prontas para uso, conforme Seção 14.

1. Implementar geração de QR Code (`qrcode`) com o payload `EST-XXXXXXX` ;
2. Implementar montagem do comando ZPL para impressoras térmicas (Seção 14.4);
3. Implementar geração de PDF em folha adesiva para impressoras comuns (Seção 14.5);
4. Implementar o canal `etiquetas:reimprimir` , reaproveitando o código existente;
5. Testar fisicamente em pelo menos um modelo de impressora térmica e uma impressora comum.

Definição de Pronto — Passo 7	Status esperado
QR Code impresso é lido corretamente por scanner físico em teste de bancada	Obrigatório
Reimpressão nunca gera um novo código para o mesmo item	Obrigatório

7.1 Envio Direto do Comando ZPL para a Impressora (Windows, via porta compartilhada)

```

import { exec } from 'node:child_process';

export function enviarZplParaImpressora(zpl: string, nomeImpressora: string) {
  const comando = `print /D:"\\\\localhost\\${nomeImpressora}" "${caminhoArquivoTemporarioZpl}"`;
  exec(comando, (erro) => {
    if (erro) throw new ErroImpressao(`Falha ao enviar ZPL: ${erro.message}`);
  });
}
```

7.2 Problemas Esperados e Correções — Passo 7

Problema	Causa provável	Correção
QR Code impresso fica ilegível para o scanner	Densidade de pontos do ZPL (<code>^BQN,2,4</code>) incompatível com a resolução DPI real da impressora térmica	Ajustar o parâmetro de magnificação do comando <code>^BQ</code> conforme o DPI da impressora (203 vs 300 DPI geram resultados diferentes) e testar fisicamente antes de padronizar
Impressão funciona no computador de desenvolvimento, mas falha na máquina do almoxarifado	Nome da impressora compartilhada configurado incorretamente ou driver não instalado na máquina de destino	Tornar o nome da impressora configurável em <code>sistema:configuracoes</code> (Seção 6.3) em vez de fixo no código
Reimpressão gera código novo por engano	Handler de reimpressão reutilizando a mesma função de criação em vez de buscar o código existente	Separar claramente <code>gerarNovaEtiqueta()</code> de <code>reimprimirEtiquetaExistente(itemId)</code> no código — nunca compartilhar a lógica de geração de código entre as duas

Testes padrão de indústria: teste automatizado decodificando a imagem do QR Code gerado (biblioteca de leitura em teste) e validando que o payload corresponde ao `codigo_etiqueta` esperado; teste manual de impressão física em bancada de homologação.

Testes de segurança e dados: teste garantindo que o payload do QR Code não expõe nenhum dado sensível (apenas o código, nunca preço ou dados de usuário); validação de que o nível de correção de erro M realmente tolera ~15% de dano físico simulado (etiqueta parcialmente coberta em teste de bancada).

PASSO 8 — Módulo de Entradas e Saídas + Controle de Concorrência

Objetivo: movimentação de estoque robusta sob uso concorrente, seguindo a Seção 11.3 e o exemplo crítico da Seção 17.1.

1. Implementar `movimentacoes:registrar-saida` com `SELECT ... FOR UPDATE` dentro de transação;
2. Implementar `movimentacoes:registrar-entrada`, incluindo o sub-fluxo de ordem de compra;
3. Implementar a tela de relatório de movimentações com todos os filtros da Seção 11.3.3;
4. Congelar o valor unitário no momento da movimentação (nunca recalcular retroativamente).

Definição de Pronto — Passo 8	Status esperado
Duas retiradas simultâneas do último item em estoque: apenas uma é aceita, a outra recebe erro de estoque insuficiente	Obrigatório
Valor histórico do relatório não muda se o preço do item for editado depois	Obrigatório

8.1 Implementação Completa da Transação de Saída (Prisma)

```
export async function registrarSaida(dto: SaidaDTO, usuarioId: number) {
  return prisma.$transaction(async (tx) => {
    const item = await tx.$queryRaw`
      SELECT quantidade_atual, preco_unitario FROM itens
      WHERE id = ${dto.itemId} FOR UPDATE
    `;
    if (item.quantidadeAtual < dto.quantidade) {
      throw new ErroEstoqueInsuficiente(
        `Estoque atual (${item.quantidadeAtual}) menor que o solicitado (${dto.quantidade})`,
      );
    }
  });
}
```

```

    });
  }
  await tx.item.update({
    where: { id: dto.itemId },
    data: { quantidadeAtual: { decrement: dto.quantidade } },
  });
  return tx.movimentacaoEstoque.create({
    data: {
      itemId: dto.itemId,
      tipoMovimento: 'SAIDA',
      subtipo: dto.subtipo,
      quantidade: dto.quantidade,
      usuarioId,
      observacoes: dto.observacoes,
      // valor congelado no momento da movimentacao:
      valorUnitarioRegistrado: item.precoUnitario,
    },
  });
}, { isolationLevel: 'ReadCommitted' });
}

```

8.2 Problemas Esperados e Correções — Passo 8

Problema	Causa provável	Correção
Deadlock ocasional sob alta concorrência (Error 1213: Deadlock found)	Duas transações bloqueando linhas de itens diferentes em ordens opostas	Padronizar a ordem de bloqueio (ex: sempre por <code>item_id</code> crescente quando uma operação envolve múltiplos itens) e implementar retry automático com backoff em caso de deadlock
Relatório mostra valor total diferente do esperado após reajuste de preço	Cálculo de valor histórico feito com o preço atual em vez do preço congelado na movimentação	Garantir que a coluna <code>valor_unitario_registrado</code> (adicionar ao schema se ausente) seja sempre usada em relatórios, nunca <code>itens.preco_unitario</code> diretamente
Transação trava indefinidamente em teste de carga	Ausência de timeout de transação configurado	Configurar <code>transactionOptions.timeout</code> no Prisma (ex: 5000ms) para falhar rápido em vez de travar a UI do operador

Testes padrão de indústria: teste de integração disparando duas transações concorrentes contra o mesmo `item_id` (teste de corrida clássico) e validando o resultado determinístico; teste de regressão do cálculo de valor histórico.

Testes de segurança e dados: teste garantindo que um Operador não consegue registrar entrada do subtipo `COMPRA` (regra de negócio da Seção 10.3); teste de integridade referencial — tentar registrar movimentação para um `item_id` inexistente deve falhar de forma controlada, não com erro não tratado.

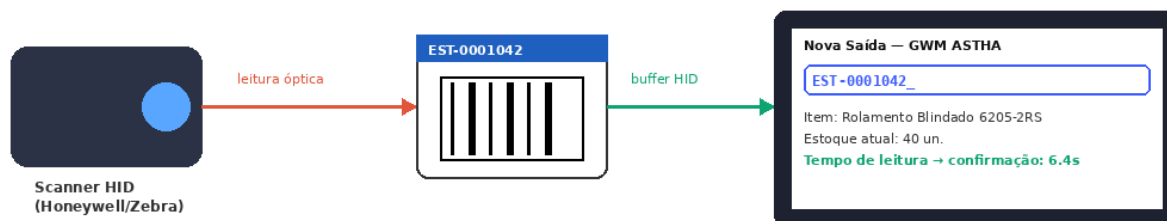
PASSO 9 — Integração com Scanner HID

Objetivo: retirada de item via leitura de QR Code funcionando de ponta a ponta (Seção 13).

1. Implementar o hook `useScannerListener` (Seção 13.1) na tela "Nova Saída";
2. Implementar o fluxo completo: leitura → busca do item → preenchimento automático → confirmação;
3. Testar com pelo menos dois modelos de scanner físico da tabela da Seção 13.2;
4. Implementar as mensagens de erro do fluxograma da Seção 12.2 (item não encontrado, estoque insuficiente).

Definição de Pronto — Passo 9	Status esperado
Retirada completa via scanner físico em menos de 10 segundos (medição cronometrada)	Obrigatório

Bancada de Teste — Leitura de QR Code via Scanner HID



Checklist de bancada:

1. Distância de leitura testada: 5cm, 15cm, 30cm
2. Ângulo de leitura testado: 0°, 15°, 30°
3. Etiqueta parcialmente danificada (15% de área) ainda legível (nível de correção M)
4. Tempo total do fluxo cronometrado (meta: < 10s)
5. Digitação manual acidental não gera falso-positivo

Figura 9 — Bancada de teste ilustrativa do fluxo scanner → buffer HID → aplicação, com checklist de validação física.

9.1 Hook Completo com Proteção Anti Falso-Positivo

```

export function useScannerListener(onCodigoLido: (codigo: string) => void, ativo: boolean) {
  useEffect(() => {
    if (!ativo) return;
    let buffer = '';
    let ultimoEvento = Date.now();

    const handler = (e: KeyboardEvent) => {
      const agora = Date.now();
      const intervalo = agora - ultimoEvento;
      ultimoEvento = agora;

      // Scanners HID digitam a ~1-10ms por caractere; humanos, >80ms.
      if (intervalo > 60 && buffer.length > 0) {
        buffer = ''; // muito lento para ser scanner - provavel digitacao manual
      }

      if (e.key === 'Enter') {
        if (/^EST-\d{7}$/.test(buffer)) {
          onCodigoLido(buffer);
        } else if (buffer.length > 0) {
          toast.error('Código não reconhecido. Use o formato EST-0000000.');
```

9.2 Roteiro de Teste de Bancada (a ser executado fisicamente)

1. Imprimir 5 etiquetas de teste em impressora real (Seção 14.4/14.5);

2. Testar leitura a 5cm, 15cm e 30cm de distância do scanner;
3. Testar leitura com ângulo de 0°, 15° e 30° em relação à etiqueta;
4. Cobrir manualmente ~15% da área do QR Code com fita adesiva e confirmar que ainda é lido (validação do nível de correção de erro M);
5. Cronometrar o tempo entre "abrir tela de saída" e "confirmação exibida na tela" em 10 tentativas, calculando a média.

9.3 Problemas Esperados e Correções — Passo 9

Problema	Causa provável	Correção
Scanner "digita" o código, mas nada acontece na tela	O campo/listener não está ativo (usuário navegou para outra aba antes de escanear)	Ativar o listener apenas quando a tela "Nova Saída" está em foco (parâmetro ativo do hook) e exibir indicador visual de "aguardando leitura"
Leitura funciona no scanner de bancada, mas falha no scanner real da fábrica	Modelo de scanner configurado para adicionar um caractere de sufixo diferente de Enter (ex: Tab)	Padronizar a configuração do scanner via seu manual/scanner de configuração (a maioria aceita reconfiguração escaneando um código de barras especial de configuração)
Leitura falha aleatoriamente em ~1 de cada 20 tentativas	Etiqueta impressa com contraste insuficiente (impressora térmica com fita/ribbon gasto)	Estabelecer rotina de manutenção preventiva da impressora (Seção 14) e considerar reduzir o intervalo de troca de ribbon
Tempo total do fluxo passa de 10s por lentidão da consulta ao banco	Ausência de índice efetivo em codigo_etiqueta ou conexão de rede lenta com o MySQL remoto	Confirmar uso do índice idx_itens_codigo (Seção 8.2) via EXPLAIN ; considerar cache local de itens mais frequentemente retirados

Testes padrão de indústria: teste E2E simulando a digitação rápida característica de um scanner (Seção 16.3); teste de usabilidade cronometrado com usuário real do perfil Operador.

Testes de segurança e dados: teste garantindo que um código escaneado que não segue o padrão **EST-** é ignorado (não gera consulta ao banco com entrada arbitrária); teste de injeção — tentar escanear/digitar uma string contendo caracteres de SQL injection e confirmar que o Prisma/ORM parametriza corretamente a consulta.

PASSO 10 — Fluxo de Aprovação de Remoção

Objetivo: implementar o fluxo de governança da Seção 11.9 e 12.3.

1. Implementar **itens:solicitar-remocao** e **itens:aprovar-remocao**;
2. Implementar notificação in-app para Analista/GL/ADM (badge de pendências);
3. Garantir que a aprovação executa, em uma única transação: **item.ativo = false**, movimentação **BAIXA_REMOCAO** e log de auditoria.

Definição de Pronto — Passo 10	Status esperado
Técnico/Operador não conseguem aprovar sua própria solicitação (mesmo com nível de acesso manipulado no cliente)	Obrigatório
Item rejeitado permanece ativo e disponível normalmente	Obrigatório

10.1 Transação Atômica de Aprovação

```
export async function aprovarRemocao(solicitacaoId: number, aprovadorId: number) {
  return prisma.$transaction(async (tx) => {
    const solicitacao = await tx.solicitacaoRemocao.findUniqueOrThrow({ where: { id: solicitacaoId } });
    if (solicitacao.solicitanteId === aprovadorId) {
```

```

    throw new ErroRegraNegocio('Um usuário não pode aprovar a própria solicitação de remoção.');
```

```

  }
  await tx.item.update({ where: { id: solicitacao.itemId }, data: { ativo: false } });
  await tx.movimentacaoEstoque.create({
    data: { itemId: solicitacao.itemId, tipoMovimento: 'SAIDA', subtipo: 'BAIXA_REMOCAO', quantidade: 0,
usuarioId: aprovadorId },
  });
  await tx.solicitacaoRemocao.update({
    where: { id: solicitacaoId },
    data: { status: 'APROVADO', aprovadorId, dataResposta: new Date() },
  });
  await registrarLogAuditoria(tx, { usuarioId: aprovadorId, acao: 'APROVAR_REMOCAO', registroId:
solicitacaoId });
  });
}

```

10.2 Problemas Esperados e Correções — Passo 10

Problema	Causa provável	Correção
Item aprovado para remoção continua aparecendo na busca de estoque	Tela de busca não filtra <code>ativo = true</code> por padrão	Aplicar filtro padrão em todas as consultas de listagem de itens, com opção explícita de "mostrar removidos" apenas para Analista/GL/ADM
Notificação de pendência não aparece para o GL, apenas para o Analista	Query de notificação filtrando por cargo exato em vez de nível de acesso mínimo	Notificar todos os usuários com <code>nivelAcesso >= 3</code> , não apenas o cargo "Analista" literal

Testes padrão de indústria: teste E2E do fluxo completo solicitação → notificação → aprovação/rejeição; teste de integração garantindo atomicidade da transação de aprovação (se o log falhar, a baixa do item também deve ser revertida).

Testes de segurança e dados: teste negativo tentando chamar `itens:aprovar-remocao` diretamente via IPC com sessão de nível Técnico — deve ser rejeitado no Main Process independentemente do que a UI permite; teste garantindo que o próprio solicitante nunca consegue aprovar sua solicitação, mesmo sendo Analista/GL/ADM.

PASSO 11 — Dashboard e KPIs

Objetivo: visão gerencial completa conforme Seção 11.4, com performance adequada mesmo em catálogos grandes.

1. Implementar o canal `dashboard:kpis` com os agregados da Seção 11.4;
2. Implementar a tabela/estratégia de cache `dashboard_cache` para os agregados pesados;
3. Construir a UI seguindo o mockup da Figura 2 (Seção 7), com os cartões, gráficos e listas de aprovações/tarefas.

Definição de Pronto — Passo 11	Status esperado
Dashboard carrega em < 1s mesmo com 100 mil itens em base de teste de carga	Obrigatório
Valor total em estoque bate exatamente com <code>SUM(quantidade_atual * preco_unitario)</code> calculado manualmente em query de auditoria	Obrigatório

11.1 Atualização Incremental do Cache (Trigger de Aplicação, não de Banco)

```

// Chamado ao final de toda transacao que altera quantidade_atual ou preco_unitario
export async function atualizarDashboardCache() {
  const agregado = await prisma.item.aggregate({

```

```

    where: { ativo: true },
    _sum: { quantidadeAtual: true },
  });
  const valorTotal = await prisma.$queryRaw<{ total: number }[]>`
    SELECT SUM(quantidade_atual * preco_unitario) as total FROM itens WHERE ativo = 1
  `;
  await prisma.dashboardCache.upsert({
    where: { id: 1 },
    update: { valorTotalEstoque: valorTotal[0].total, atualizadoEm: new Date() },
    create: { id: 1, valorTotalEstoque: valorTotal[0].total },
  });
}

```

11.2 Problemas Esperados e Correções — Passo 11

Problema	Causa provável	Correção
Valor do Dashboard "atrasado" em relação ao estoque real	Cache atualizado apenas por job agendado (ex: a cada 5 min), sem invalidação imediata em movimentações críticas	Disparar <code>atualizarDashboardCache()</code> de forma síncrona ao final de toda transação de entrada/saída/aprovação, além do job agendado como rede de segurança
Gráfico de "Top 10 itens mais retirados" muito lento em catálogos grandes	Consulta agregando toda a tabela <code>movimentacoes_estoque</code> sem filtro de período	Limitar a consulta a uma janela de tempo (ex: últimos 90 dias) por padrão, com opção de expandir o período mediante novo carregamento

Testes padrão de indústria: teste de performance/carga (ex: k6 ou script customizado) populando 100 mil itens e medindo tempo de resposta do canal `dashboard:kpi`; teste de regressão numérica comparando o KPI exibido com uma query SQL de verificação independente.

Testes de segurança e dados: teste garantindo que o ranking de "usuários que mais retiram" (Seção 11.4) só é visível para Analista/GL/ADM, nunca para Operador/Técnico — mitigando uso do dado para fins de vigilância não autorizada.

PASSO 12 — Exportação de Relatórios (CSV/XLSX)

Objetivo: exportação confiável e íntegra conforme Seção 15.

1. Implementar `relatorios:exportar-csv` e `relatorios:exportar-xlsx`;
2. Integrar com `dialog.showSaveDialog` nativo do Electron;
3. Aplicar a convenção de nomenclatura de arquivo da Seção 15.3.

Definição de Pronto — Passo 12	Status esperado
Arquivo XLSX abre corretamente no Excel com acentuação e formatação de moeda preservadas	Obrigatório
Arquivo CSV abre sem corrupção de caracteres especiais no Excel do Windows (BOM presente)	Obrigatório

12.1 Problemas Esperados e Correções — Passo 12

Problema	Causa provável	Correção
Acentuação corrompida ao abrir CSV no Excel ("AÃ&Ã£o" em vez de "Ação")	Arquivo salvo sem BOM UTF-8	Sempre prefixar o conteúdo com <code>'\uFEFF'</code> antes de escrever o arquivo (Seção 15.1)
	Geração do XLSX rodando no mesmo processo/thread da UI	

Exportação de 50 mil linhas trava a interface por vários segundos

Mover a geração para o Main Process (já é o padrão arquitetural) e exibir barra de progresso via evento IPC incremental

Coluna de valor monetário aparece como texto, não como número formatado

Estilo `numFmt` não aplicado corretamente na célula gerada pelo `exceljs`

Confirmar que o valor é escrito como `number`, não `string`, antes de aplicar o estilo de formatação de moeda

Testes padrão de indústria: teste automatizado abrindo o arquivo gerado com uma biblioteca de leitura (ex: `exceljs` no próprio teste) e validando conteúdo linha a linha contra os dados de origem.

Testes de segurança e dados: teste garantindo que um Técnico só exporta suas próprias movimentações (filtro de `usuario_id` forçado no backend, não confiável apenas na UI); log de auditoria `EXPORTAR_RELATORIO` gravado a cada exportação, incluindo os filtros aplicados.

PASSO 13 — Visualizador 3D e Gestão de Manuais

Objetivo: funcionalidades de suporte técnico ao item, conforme Seções 11.7 e 11.8.

- Integrar three.js com os três loaders (`OBJLoader`, `STLLoader`, `ThreeMFLoader`);
- Implementar upload/associação de manuais (PDF ou link) com visualização embutida via `pdf.js`.

Definição de Pronto — Passo 13	Status esperado
Arquivo 3D de até 30MB carrega e permite rotação/zoom sem travar a interface	Obrigatório
PDF de manual é visualizado sem sair da aplicação	Obrigatório

13.1 Problemas Esperados e Correções — Passo 13

Problema	Causa provável	Correção
Modelo <code>.stl</code> carrega totalmente cinza, sem qualquer cor	Comportamento esperado — <code>.stl</code> é geometria pura, sem material embutido	Aplicar um material padrão (<code>MeshStandardMaterial</code>) com cor neutra e boa iluminação de cena; documentar essa limitação para o usuário final
Aplicação congela por 2–3 segundos ao abrir um arquivo 3D grande	Parsing do arquivo rodando de forma síncrona na thread principal do Renderer	Mover o parsing para um <code>Web Worker</code> e exibir spinner de carregamento

Testes padrão de indústria: teste manual de carregamento com arquivos de exemplo em cada formato suportado (`.obj`, `.stl`, `.3mf`); teste de performance validando que o carregamento não bloqueia a UI (uso de worker/carregamento assíncrono).

Testes de segurança e dados: teste de upload de link de manual validando que apenas URLs com protocolo `https://` são aceitas (mitigação de `javascript://` / `file://` como vetor de ataque); sanitização do nome de arquivo do PDF antes de salvar em `storage/manuais/`.

PASSO 14 — Aplicação do Sistema de Design (UI/UX)

Objetivo: unificar visualmente todas as telas conforme os tokens da Seção 7, encerrando o ciclo funcional com uma UI coesa.

- Extrair os tokens de cor/tipografia da Seção 7.1/7.2 para um arquivo central Tailwind (`tailwind.config.js`);
- Revisar cada tela construída nos passos anteriores contra o checklist de acessibilidade da Seção 7.4;
- Padronizar componentes reutilizáveis (cartão glass, badge de status, avatar) em uma biblioteca interna de componentes.

Definição de Pronto — Passo 14	Status esperado
Nenhuma tela usa cor fora da paleta definida (validado por lint visual/revisão de design)	Obrigatório
Contraste mínimo AA validado em todas as telas principais (ferramenta automatizada, ex: axe-core)	Obrigatório

14.1 Configuração de Tokens no Tailwind

```
// tailwind.config.js
module.exports = {
  theme: {
    extend: {
      colors: {
        primary: '#3B5BFD',
        secondary: '#7C3AED',
        success: '#0EA472',
        danger: '#E0563B',
        warning: '#C98A1A',
        sidebar: '#151A23',
      },
      backgroundImage: {
        'gwm-gradient': 'linear-gradient(135deg, #D6E2FC 0%, #E9D6F7 45%, #F7DEE8 75%, #FFE7D1 100%)',
      },
    },
  },
};
```

14.2 Problemas Esperados e Correções — Passo 14

Problema	Causa provável	Correção
Teste automatizado de acessibilidade (axe-core) reporta contraste insuficiente em textos sobre o gradiente	Texto cinza claro posicionado sobre a área mais clara do gradiente (próxima ao <code>--bg-gradient-end</code>)	Restringir texto livre apenas sobre os cartões "glass" (fundo branco 82% opaco), nunca diretamente sobre o gradiente de fundo

Testes padrão de indústria: teste automatizado de acessibilidade com `axe-core` integrado ao Playwright, rodando em cada tela principal; revisão visual manual (QA) contra os mockups das Figuras 2 e 4.

Testes de segurança e dados: nenhum teste de segurança específico de UI nesta etapa, além de confirmar que nenhum dado sensível (senha, token) é exibido inadvertidamente em nenhum componente visual (ex: campos de debug esquecidos em produção).

PASSO 15 — Auditoria e Logs Imutáveis

Objetivo: garantir que toda ação relevante do sistema (Seção 18.1) já esteja gerando log corretamente, revisando retroativamente os passos anteriores.

- 1. Auditar cada canal IPC implementado nos Passos 3–13 e confirmar que gera o log correspondente;
- 2. Implementar a tela de consulta de auditoria (GL/ADM) com os filtros necessários;
- 3. Aplicar os privilégios restritos de banco de dados definidos na Seção 18.1 no ambiente de produção.

Definição de Pronto — Passo 15	Status esperado
100% das ações da tabela da Seção 18.1 geram log correspondente (checklist manual de auditoria de código)	Obrigatório

Obrigatório

Usuário de aplicação em produção não possui UPDATE/DELETE em `logs_auditoria` (validado em ambiente real, não apenas em teste)

15.1 Problemas Esperados e Correções — Passo 15

Problema	Causa provável	Correção
Alguns handlers IPC não geram log, mesmo sendo ações sensíveis	Log implementado manualmente em cada handler, com risco de esquecimento	Refatorar para um middleware/decorator central que envolve todo handler marcado como <code>@auditavel</code> , reduzindo dependência de disciplina manual
Aplicação em produção ainda consegue fazer UPDATE em <code>logs_auditoria</code>	Script de <code>GRANT</code> / <code>REVOKE</code> da Seção 18.1 nunca foi executado no servidor real, apenas documentado	Incluir a execução do script de privilégios como item obrigatório do checklist de deploy (Anexo D), não apenas como referência de documentação

Testes padrão de indústria: teste de integração percorrendo cada handler IPC crítico e verificando a criação do log correspondente após a execução; revisão de código dedicada (code review de segurança) cobrindo 100% dos canais da tabela da Seção 6.3.

Testes de segurança e dados: tentativa manual e automatizada de alterar/apagar um registro de log usando as credenciais de produção deve falhar; teste confirmando que os campos `dadosanteriores` / `dadosnovos` nunca contêm hash de senha ou outro dado sensível não necessário à auditoria.

PASSO 16 — Suíte de Testes Completa e Cobertura

Objetivo: consolidar a estratégia de testes da Seção 16 em uma suíte executável de ponta a ponta, antes do hardening final.

1. Revisar e completar a cobertura de testes unitários em `services/` até atingir $\geq 80\%$;
2. Escrever/consolidar os testes E2E dos fluxos críticos (login, cadastro, retirada, aprovação, exportação);
3. Configurar relatório de cobertura automatizado no pipeline de CI (Seção 21.1), falhando o build se a cobertura cair abaixo do mínimo.

Definição de Pronto — Passo 16	Status esperado
Cobertura de testes unitários $\geq 80\%$ em <code>services/</code>	Obrigatório
Todos os testes obrigatórios da Seção 16.4 passando no CI	Obrigatório
Nenhum teste "flaky" (instável) tolerado — taxa de falha intermitente = 0% em 10 execuções consecutivas	Obrigatório

16.1 Configuração de Cobertura Mínima (Vitest)

```
// vitest.config.ts
export default defineConfig({
  test: {
    coverage: {
      provider: 'v8',
      thresholds: {
        lines: 80,
        functions: 80,
        branches: 70,
        statements: 80,
      },
    },
    exclude: ['**/*.d.ts', 'src/pages/**'], // UI cobrimos via E2E, nao unitario
  },
});
```

```
} ,  
},  
});
```

16.2 Problemas Esperados e Correções — Passo 16

Problema	Causa provável	Correção
Teste E2E de retirada via scanner falha intermitentemente no CI (mas não localmente)	Condição de corrida entre o carregamento assíncrono do banco de teste e a simulação de digitação do Playwright	Adicionar espera explícita por um estado observável (ex: elemento "aguardando leitura" visível) em vez de <code>sleep</code> fixo
Cobertura cai de 82% para 76% após um PR que "só mudou a UI"	PR incluiu, sem perceber, uma nova função utilitária em <code>services/</code> sem teste correspondente	Gate de CI bloqueando o merge (Seção 21.1) — o próprio processo já pega isso; a correção é simplesmente escrever o teste faltante antes de mergear

Testes padrão de indústria: execução completa da pirâmide de testes da Seção 16.1 (unitário → integração → E2E); gate de qualidade no CI bloqueando merge se a cobertura cair.

Testes de segurança e dados: inclusão de testes negativos de RBAC para **cada** canal IPC listado na Seção 6.3 (não apenas os críticos) — cada canal deve ter ao menos um teste comprovando que um nível de acesso insuficiente é rejeitado.

PASSO 17 — Hardening de Segurança e Validação OWASP ASVS

Objetivo: validação final de segurança antes de qualquer implantação real, seguindo o checklist da Seção 9.5 e o modelo de ameaças da Seção 6.5.

1. Executar revisão manual do checklist OWASP ASVS (Seção 9.5) item por item, documentando evidência de conformidade;
2. Executar `npm audit` / Dependabot e resolver todas as vulnerabilidades de severidade alta/crítica em dependências;
3. Realizar um teste de penetração interno básico: tentativa de bypass de RBAC, injeção de dados, manipulação de payload IPC via ferramentas de debug;
4. Revisar a configuração de CSP e as flags de segurança do Electron (Seção 9.4) em build de produção (não apenas em desenvolvimento).

Definição de Pronto — Passo 17	Status esperado
100% dos itens do checklist da Seção 9.5 com evidência documentada	Obrigatório
Zero vulnerabilidade de severidade alta/crítica em dependências (<code>npm audit</code>)	Obrigatório
Nenhuma das 6 categorias do modelo STRIDE (Seção 6.5) com mitigação pendente	Obrigatório

Checklist de Pentest Interno — GWM ASTHA (execução manual por sprint de hardening)

Vetor de ataque testado	Procedimento	Resultado esperado
Elevação de privilégio via IPC	Chamar canal com sessão nível Técnico manipulada	Bloqueado 
Path traversal em upload	Tentar salvar manual como '..\..\windows\system32\x.pdf'	Bloqueado 
SQL Injection via campo de busca	Buscar item com payload ''; DROP TABLE itens; --"	Bloqueado (Prisma parametrizado) 
Bypass de bloqueio de conta	6ª tentativa de login simultânea em duas janelas	Bloqueado 
Senha em texto plano em log	grep -R 'senha' storage/logs/*.log	Nenhuma ocorrência 
DevTools acessível em produção	Atalho Ctrl+Shift+I em build de produção	Desabilitado 
Timing attack no login	Medir tempo de resposta: crachá válido vs inválido	Diferença < 5ms 
Exfiltração via export de relatório	Técnico tenta exportar movimentações de outro usuário	Bloqueado (filtro forçado no backend) 

Figura 10 — Checklist de pentest interno estruturado por vetor de ataque (execução manual a cada sprint de hardening).

17.1 CSP de Produção — Exemplo

```
<meta http-equiv="Content-Security-Policy" content="
  default-src 'self';
  script-src 'self';
  style-src 'self' 'unsafe-inline';
  img-src 'self' data;;
  connect-src 'self';
  object-src 'none';
">
```

17.2 Problemas Esperados e Correções — Passo 17

Problema	Causa provável	Correção
<code>npm audit</code> reporta vulnerabilidade em dependência transitiva sem correção disponível	Biblioteca de terceiro nível não mantida ativamente	Avaliar substituição da dependência direta que a importa, ou aplicar <code>overrides</code> / <code>resolutions</code> no <code>package.json</code> forçando uma versão corrigida da transitiva
CSP bloqueia o carregamento do visualizador 3D (three.js via CDN)	Diretiva <code>script-src 'self'</code> impede scripts externos	Empacotar o three.js localmente no bundle da aplicação em vez de referenciar CDN — mais seguro e mais alinhado ao uso 100% offline do sistema
DevTools ainda abre com <code>Ctrl+Shift+I</code> mesmo em build "de produção"	Variável de ambiente <code>NODE_ENV</code> não configurada corretamente no processo empacotado	Validar explicitamente o modo de build (não confiar apenas em <code>NODE_ENV</code>) e remover o listener do atalho de DevTools no código de produção

Testes padrão de indústria: scan automatizado de dependências (`npm audit` , Snyk ou similar) integrado ao CI; revisão de código de segurança dedicada (par de desenvolvedores, "four eyes principle") em todos os módulos de autenticação/RBAC.

Testes de segurança e dados: pentest interno estruturado cobrindo: (1) tentativa de elevação de privilégio via manipulação de payload IPC; (2) tentativa de acesso a arquivo fora de `storage/` via path traversal; (3) tentativa de bypass do bloqueio de conta via múltiplas sessões simultâneas; (4) inspeção de memória/processo em busca de senha em texto plano residual; (5) validação de que o build de produção não expõe DevTools do Electron.

PASSO 18 — CI/CD, Empacotamento e Distribuição

Objetivo: pipeline de build e distribuição confiável, conforme Seção 21.

1. Finalizar o pipeline de CI/CD completo (lint → test → build → package), conforme Seção 21.1;
2. Configurar `electron-builder` para gerar o instalador NSIS assinado com certificado interno;
3. Publicar o instalador no repositório interno (Nexus/Artifactory ou pasta de rede);
4. Documentar o processo de rollback (reinstalação da versão anterior) em caso de falha crítica pós-release.

Definição de Pronto — Passo 18	Status esperado
Pipeline gera instalador funcional a cada push na branch principal	Obrigatório
Instalador assinado não gera alerta bloqueante do SmartScreen em máquina de teste limpa	Recomendado

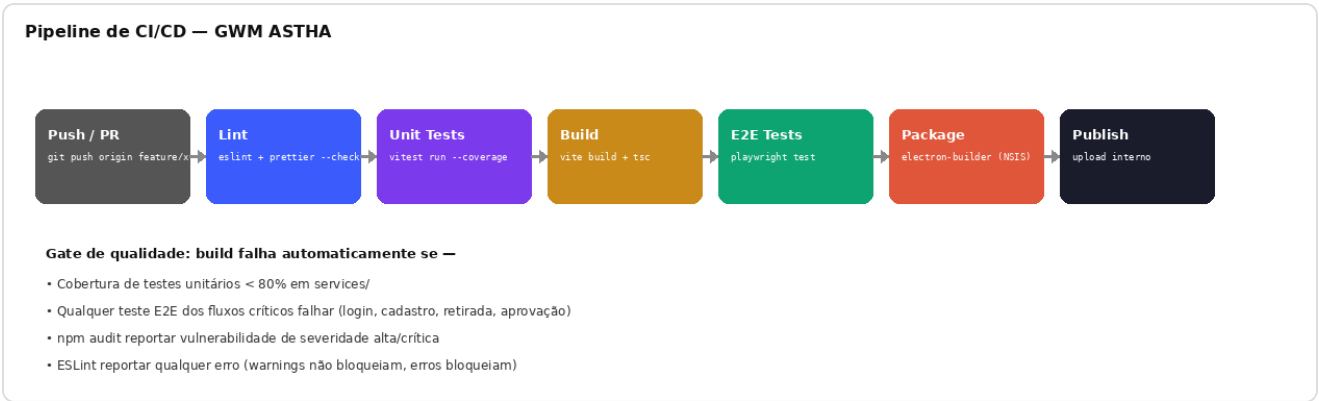


Figura 11 — Pipeline de CI/CD do GWM ASTHA, com os gates de qualidade que bloqueiam o build automaticamente.

18.1 Problemas Esperados e Correções — Passo 18

Problema	Causa provável	Correção
Instalador gerado no CI não abre na máquina de teste ("aplicativo corrompido")	Certificado de assinatura de código expirado ou ausente no ambiente de CI	Automatizar a verificação de validade do certificado antes do step de assinatura, falhando o pipeline com mensagem clara em vez de gerar um instalador não assinado silenciosamente
Build funciona no CI, mas falha ao empacotar localmente ("works on my machine" às avessas)	Diferença de versão do Node.js entre o ambiente de CI e a máquina do desenvolvedor	Fixar a versão do Node via arquivo <code>.nvmrc</code> / <code>engines</code> no <code>package.json</code> e replicar exatamente no runner de CI

Testes padrão de indústria: instalação limpa do instalador gerado em máquina virtual "de fábrica" (sem Node/ dependências pré-instaladas), validando que o app abre e conecta ao banco corretamente.

Testes de segurança e dados: verificação de que o instalador não inclui arquivos `.env`, chaves de assinatura ou dados de seed de desenvolvimento; validação de assinatura digital do executável antes da distribuição.

PASSO 19 — Deploy Piloto, Treinamento e Go-Live

Objetivo: implantação controlada na sala de estoque real, seguindo o plano de treinamento da Seção 23.

1. Instalar em uma máquina piloto seguindo o checklist do Anexo D;
2. Executar migração/carga inicial dos itens existentes (planilha legada → importação estruturada, se aplicável);
3. Treinar os usuários conforme a trilha por perfil da Seção 23.1;
4. Acompanhar os primeiros 5 dias de uso real com suporte próximo (hypercare), monitorando logs de erro (`electron-log`);
5. Congelar a versão de produção após estabilização e formalizar o encerramento do projeto piloto.

Definição de Pronto — Passo 19	Status esperado
Zero incidente crítico (perda de dados, indisponibilidade) nos primeiros 5 dias de uso real	Obrigatório
100% dos usuários do piloto treinados e com aceite de termos registrado	Obrigatório

19.1 Problemas Esperados e Correções — Passo 19

Problema	Causa provável	Correção
Usuários reais retiram itens de forma diferente do previsto no fluxograma (ex: retirando para terceiros com frequência)	Levantamento de requisitos não capturou 100% das variações reais de operação do almoxarifado	Ajustar o campo "responsável pela retirada" (Seção 11.3.1) para ser mais proeminente na UI, já que esse comportamento se mostrou comum na prática
Migração da planilha legada gera itens duplicados	Planilha original já tinha duplicatas não percebidas por nome similar (ex: "Parafuso M6" vs "Parafuso m6 ")	Executar um script de deduplicação com normalização de texto (trim, lowercase, remoção de acentos) antes da importação definitiva, com revisão manual dos casos ambíguos

Testes padrão de indústria: teste de aceitação do usuário (UAT) formal com os perfis reais (Operador, Técnico, Analista, GL) validando os fluxos do dia a dia antes da liberação definitiva.

Testes de segurança e dados: validação final de que o backup diário (Seção 19.2) está de fato rodando e gerando arquivo íntegro na máquina de produção; teste de restauração completo em ambiente de contingência antes de declarar o go-live estável.

PASSO 20 — Pós Go-Live: Monitoramento e Ciclo de Melhoria

Objetivo: manter a qualidade do sistema ao longo do tempo, evitando a degradação típica de sistemas "terminados e esquecidos".

1. Estabelecer rotina mensal de revisão de logs de erro e de auditoria;
2. Estabelecer rotina trimestral de teste de restauração de backup (Seção 19.2);
3. Estabelecer rotina semestral de re-execução do checklist OWASP ASVS (Seção 9.5), já que novas dependências e novas funcionalidades podem introduzir regressões de segurança;
4. Manter esta documentação viva — toda nova funcionalidade relevante deve atualizar as Seções correspondentes.

Definição de Pronto — Passo 20 (contínuo)	Status esperado
Nenhum backup falho não detectado por mais de 24h (alerta configurado)	Obrigatório
Revisão de segurança semestral documentada e assinada pelo responsável de TI	Obrigatório

20.1 Problemas Esperados e Correções — Passo 20

Problema	Causa provável	Correção
Backup roda, mas ninguém percebe quando ele começa a falhar silenciosamente	Ausência de alerta automatizado — a rotina só é verificada "de vez em quando"	Implementar um script de verificação pós-backup que confere o tamanho/integridade do arquivo gerado e envia alerta (e-mail interno ou log destacado) em caso de falha ou arquivo suspeito de tamanho zero
Débito técnico se acumula silenciosamente ao longo dos meses	Nenhuma métrica objetiva de qualidade de código é acompanhada após o go-live	Adotar ferramenta de análise estática contínua (SonarQube ou similar) com meta de "nunca piorar" a nota de qualidade a cada release

Testes padrão de indústria: auditoria de código periódica (ex: SonarQube ou similar) para detectar degradação de qualidade/dívida técnica ao longo do tempo.

Testes de segurança e dados: reavaliação periódica do modelo de ameaças STRIDE (Seção 6.5) sempre que uma nova integração ou funcionalidade for adicionada, e reexecução completa do pentest interno (Passo 17) a cada major release.

Resumo do tutorial: 20 passos, cada um com sua Definição de Pronto, testes de indústria, testes de segurança/dados e uma tabela de problemas esperados com correção pertinente — do primeiro `git init` ao ciclo contínuo de manutenção pós-produção. Nenhum passo deve ser pulado; nenhuma Definição de Pronto deve ser flexibilizada "só essa vez".

27. Considerações Finais

Esta documentação — em sua revisão 2.1 — cobre a espinha dorsal técnica, funcional, visual, legal, operacional e agora também o passo a passo completo de construção do GWM ASTHA (Seção 26). Pontos de atenção para quem for manter o código e o produto no futuro:

- **Nunca** contornar o RBAC "temporariamente" para debug em produção — é assim que auditorias viram piada e que perdas patrimoniais viram surpresa;
- Toda alteração de schema deve passar por migration versionada, nunca `ALTER TABLE` manual direto em produção;
- O `codigo_etiqueta` é a chave de rastreabilidade física de todo o sistema — jamais permitir edição manual dele após gerado;
- Ao expandir para múltiplas estações, revisar o nível de isolamento de transação sob carga real antes de assumir que `READ COMMITTED` (padrão do InnoDB) é suficiente;
- Toda nova funcionalidade relevante deve atualizar esta documentação — versão viva, não um documento estático esquecido em uma pasta de rede;
- Os Termos de Uso e a Política de Dados (Seções 3 e 4) devem ser revisados juridicamente antes da primeira publicação oficial, já que este documento fornece a base técnica, mas não substitui validação formal do jurídico/RH da empresa;
- O Sistema de Design (Seção 7) deve ser tratado como uma referência viva — ao evoluir a UI, atualizar os tokens de design documentados aqui para manter consistência entre código e documentação.

Resumo executivo: o GWM ASTHA entrega rastreabilidade total, controle financeiro, identificação física padronizada via QR Code, governança de acesso por cargo e conformidade com LGPD — substituindo de forma definitiva o modelo de planilhas soltas por um sistema auditável, seguro e alinhado às melhores práticas de mercado para software desktop corporativo.

Fim da documentação técnica — Sistema de Gerenciamento de Estoque Interno (GWM ASTHA) — Revisão 2.1